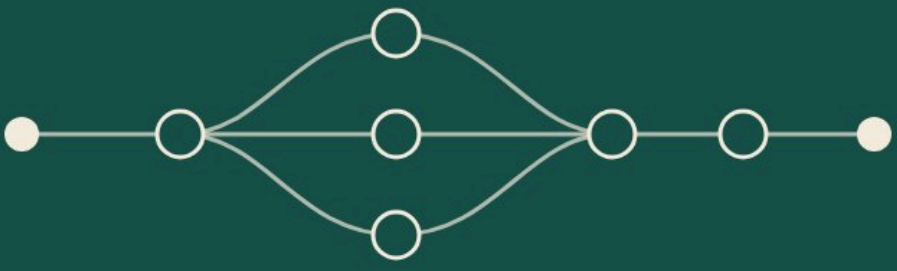
Building Scout

A plain-English guide to AI agents, and the tools that build them



Building Scout

A plain-English guide to AI agents, and the tools that build them

Christopher Haworth

Contents

Before you begin

Part One: Foundations

Chapter 1. What an agent actually is

Chapter 2. Setting up your workshop

Chapter 3. Talking to Claude well

Part Two: Working with Claude Code

Chapter 4. Building with Claude Code

Chapter 5. Giving Claude a memory

Chapter 6. Checks that can say no

Part Three: Building the agent

Chapter 7. Writing tools for agents

Chapter 8. A team of researchers

Chapter 9. Drawing the map

Part Four: Scaling your workshop

Chapter 10. Helpers with their own desks

Chapter 11. Teaching Claude your recipes

Chapter 12. Plugging into the world

Part Five: Running on its own

Chapter 13. Work that outlasts a conversation

Chapter 14. The agent in a loop

Chapter 15. Shipping Scout

Appendices

Appendix A. Answers to the tests

Appendix B. Glossary

Appendix C. When things go wrong

Appendix D. Scout's complete source
Appendix E. Sources and further reading

Before you begin

By the last page of this book you will have built Scout, a small web application that answers hard questions the way a good research assistant would. You type a question. Scout breaks it into parts, sends a team of AI researchers out onto the web in parallel, gathers what they find, writes a report with numbered sources, checks its own work, and hands it back to you. You will be able to put it on the internet with a password and use it every day.

You will not write most of the code yourself. You will direct an AI coding tool, Claude Code, to write it, and you will learn enough about what it is doing to steer it, check it and fix it. That is the real skill this book teaches: how to work with AI agents, and how to build one.

Who this book is for

This book is for people who have never programmed, or who tried once and gave up. If you can install an app, copy and paste, and follow a recipe, you can finish it. Where a technical word is unavoidable, it is explained the first time it appears and collected in the glossary at the back.

It is also for people who have heard the words *agent*, *MCP*, *subagent* and *harness* in meetings and would like to know what they mean, by building something real with each of them.

What you will build, chapter by chapter

Part	Chapters	What Scout gains
One: Foundations	1 to 3	You learn what an agent is, set up your computer, and write Scout's first prompt

Part	Chapters	What Scout gains
Two: Working with Claude Code	4 to 6	Scout becomes a working web page; Claude Code learns your project and gets safety checks
Three: Building the agent	7 to 9	Scout gets tools, a team of parallel researchers, and a map of how work flows
Four: Scaling your workshop	10 to 12	You add helper agents, reusable recipes and connections to other software
Five: Running on its own	13 to 15	Scout gets new features overnight, tunes its own prompts, and goes live

How each chapter works

Every chapter has three kinds of material.

- **The idea.** A short explanation of one concept, taken from the source material Anthropic and others have published, in plain language.
- **Build it.** Numbered steps you follow at your keyboard. Most steps ask you to type an instruction to Claude Code. The exact words are shown in a box. You can change them, but the first time through it is best to copy them.
- **Check your understanding.** A short test at the end of the chapter: a few questions and one hands-on exercise. Answers are in Appendix A. Do the test before moving on; the later chapters assume you have the earlier ones in your hands, not just in your head.

Claude Code does not write exactly the same code twice. Your files will differ a little from mine. That is normal. Appendix D contains the complete reference version of Scout, file by file, and every chapter ends with a checkpoint that tells you how to compare your work with it. If you get badly stuck, you can always ask Claude Code to bring a file into line with the reference.

What you need

- A computer running macOS, Windows 10 or 11, or Linux, with about 2 GB of free space.
- A Claude subscription (Pro or Max) or an Anthropic Console account, to use Claude Code.
- An Anthropic API account with a small amount of credit. Scout calls Claude directly, and each research question costs a few pence to a few tens of pence depending on how hard it is. Ten pounds or dollars of credit is plenty for the whole book.
- A free GitHub account, for saving your work and for putting Scout online at the end.
- Patience for the first two chapters. Setting up is the dullest part, and it is over quickly.

A note on things changing

This book was written in November 2026. The tools it uses are updated every few weeks. Menus move, commands get new options, model names change. The ideas underneath change much more slowly. Where something on your screen does not match the book, look for the nearest equivalent, and check the official documentation listed in Appendix E. The book tells you where each idea came from so you can always go back to the source.

About the author

Christopher Haworth is a product manager and AI enthusiast.

Part One: Foundations

Chapter 1. What an agent actually is

An agent is a language model that uses tools in a loop until a job is done. That is the whole definition, and most of this book is about the three words that do the work in it: *tools*, *loop* and *done*.

Start with the thing at the centre. A large language model, such as Claude, is a program that has read an enormous amount of text and learned to continue it. You give it some text, called a **prompt**, and it writes what plausibly comes next. Everything it can see at one moment, your prompt plus its own reply so far, sits in its **context window**, a working memory measured in **tokens** (roughly three-quarters of a word each). When the conversation ends, that memory is gone.

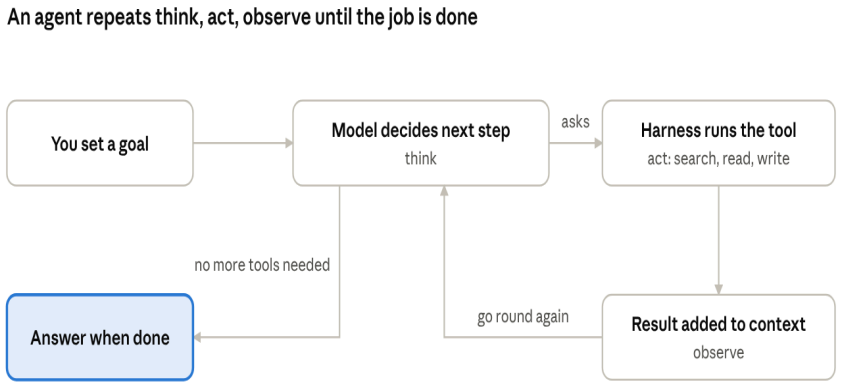
On its own, a model can only talk. It cannot look anything up, it cannot press buttons, and it cannot check whether what it said was true. Ask it today's weather and it will either decline or guess.

The augmented model

Anthropic's engineering team, in their article *Building effective agents*, describe the basic building block of every agent system as an **augmented LLM**: a model given three extra powers.

- **Retrieval**: it can ask for documents or search results to be put into its context.
- **Tools**: it can ask for an action to be taken, such as running a search, reading a file or sending an email, and see the result.
- **Memory**: it can write notes that outlast a single conversation.

The important word is *ask*. The model never runs anything itself. It writes a structured request, "call the tool named `web_search` with the query `heat pump running costs UK`", and a piece of ordinary software, which this book calls the **harness**, carries it out and hands back the result. You will write that harness for Scout in Chapter 7. It is about forty lines.



the agent loop · think, act, observe

The model only ever asks; the harness acts, and the result goes back into the model's context for its next decision. The loop ends when the model judges that no more tools are needed.

Workflows and agents

The same article draws a line that this book uses throughout. Both sides of the line are useful.

A **workflow** is a system where *you* decide the steps in advance and the model fills in each one. First summarise the document, then translate the summary, then check the translation. The path is fixed in code.

An **agent** is a system where the *model* decides the steps. You give it a goal and some tools, and it chooses what to do next based on what happened last: search, read the result, decide the result is thin, search

again with different words, then stop when it has enough. The path is discovered as it goes.

Workflows are predictable and cheap. Agents are flexible and expensive. The article's advice is to use the simplest thing that works, and to add autonomy only when a fixed path genuinely cannot do the job. Research is one of the clearest cases where it cannot: you do not know in advance which searches will pay off.

The five patterns

Between a single prompt and a fully autonomous agent, the article names five patterns that turn up again and again. You will use four of them in Scout.

Pattern	What it is	Everyday picture	Where Scout uses it
Prompt chaining	One model call after another, each using the last one's output	An assembly line	Plan, then research, then write, then review
Routing	A first call sorts the input and sends it to the right specialist	A hospital receptionist	Not used; a good exercise for later
Parallelisation	Several calls run at the same time on separate pieces, then the results are combined	A team splitting a reading list	Several researchers search at once
Orchestrator-workers	A lead model decides how to split the work and hands pieces to workers	A newspaper editor assigning stories	The planner decides the research tasks
Evaluator-optimizer	One call produces, another judges, and the first tries again with the feedback	A writer and an editor	The reviewer sends weak reports back to the writer

The researchers themselves are true agents: each one decides its own searches, in a loop, until it judges its task finished. Scout as a whole is a workflow built out of agents, which is how most real systems are

built.

Three principles worth keeping

The article closes with three principles. They will come back in almost every chapter.

1. **Keep it simple.** Add a step or an agent only when you can show it makes things better. Chapter 14 gives you a way to show it.
2. **Make the agent's thinking visible.** If you can see the plan and each step, you can tell where it went wrong. Scout's web page shows its progress for this reason.
3. **Design the agent's tools as carefully as its prompts.** The authors call this the agent-computer interface, and say they spent more time improving tools than prompts when building a coding agent. Chapter 7 is entirely about this.

Why multiple agents for research?

Anthropic built Claude's own Research feature as a lead agent directing a team of searching subagents, and wrote up what they learned in *How we built our multi-agent research system*. Their internal evaluation found the team approach beat a single agent working alone by about 90 percent on research questions. The reason is simple once you see it: each researcher has its own fresh context window, so the team as a whole can read far more than any one agent could hold in its head. The cost is just as clear: they report that multi-agent systems use around fifteen times as many tokens as an ordinary chat. Scout copies this design at a small scale and keeps strict limits on effort, so a question costs pence rather than pounds.

Check your understanding

1. In one sentence, what is the difference between a workflow and an agent?

2. A model "calls a tool". Who actually runs the tool?
3. Name the three augmentations that turn a plain model into an augmented LLM.
4. Which pattern is at work when a reviewer sends a report back to the writer with a list of problems?
5. Give one reason a team of research agents can outperform a single agent, and one reason it might not be worth it.

Exercise. Think of a task from your own work, such as answering a customer email or preparing a weekly report. Write down the steps as a workflow. Then mark the one step where a fixed path would break down and the model would need to decide for itself. That step is where an agent belongs.

Sources: Building effective agents; How we built our multi-agent research system.

Chapter 2. Setting up your workshop

This chapter installs four things, gets you an API key and opens your first conversation with Claude Code. Allow an hour. Nothing here is difficult, but each step depends on the one before, so do them in order.

The terminal, briefly

The **terminal** is a window where you type commands instead of clicking. Claude Code lives there, so you need to be able to open one and do four things in it.

- **macOS**: press Cmd + Space, type *Terminal*, press Enter.
- **Windows**: press the Windows key, type *PowerShell*, press Enter.
- **Linux**: you know where it is.

The four things:

Command	What it does	Windows PowerShell equivalent
`pwd`	Shows which folder you are in	same
`ls`	Lists the files in this folder	same (or `dir`)
`cd foldername`	Moves into a folder; `cd ..` moves back out	same
`mkdir foldername`	Makes a new folder	same

When this book shows a command in a box, type it exactly, press Enter, and wait for the prompt to come back before typing the next one. If a command prints an error, read the last line first; it usually says what is wrong.

Build it: install the tools

1. **Install Python.** Python is the language Scout is written in. Go to python.org, download the latest 3.x installer for your system and run it. On Windows, tick the box that says *Add python.exe to PATH* before clicking Install. Then, in a new terminal window:

```
```

On Windows type `python --version` instead. You should see `Python 3.12` or later. From here on, Windows readers type `python` wherever the book says `python3`.

2. **Install Git.** Git keeps a history of every change to your project so you can always go back. On macOS, run `xcode-select --install` and accept the prompt. On Windows, install *Git for Windows* from git-scm.com, keeping the default options. Check it with `git --version`.

3. **Install a text editor.** You will want to look at files now and then. Visual Studio Code (code.visualstudio.com) is free and works on every system. Any editor will do.

4. **Install Claude Code.** Claude Code is Anthropic's coding agent: Claude, running in your terminal, with tools to read and write files and run commands in the folder you start it in. On macOS, Linux or WSL:

```
```

On Windows, in PowerShell:

```
```

The installer shows no progress bar while it downloads, so give it a minute. Then **open a new terminal window** and check:

```
```

If you see a version number, it worked. If your terminal says `claude` is not found, close every terminal window, open a fresh one and try again. If it still fails, run `claude doctor` and follow what it says.

Build it: get an API key

Claude Code uses your Claude subscription. Scout itself is a separate program that will call Claude directly over the internet, and for that it needs an **API key**: a long secret password that identifies your account and lets Anthropic charge it.

1. Go to platform.claude.com and sign in or create an account.
2. Under **Billing**, add a small amount of credit. While you are there, set a monthly spending limit you are comfortable with. This is your safety net.
3. Under **API keys**, create a key named `scout`. Copy it somewhere safe immediately; you will not be shown it again. It starts with `sk-ant-`.
4. Scout's researchers use Claude's built-in web search. Your organisation's settings in the Console must allow web search; if a setting for it is switched off, switch it on.

Treat the key like a bank card. Anyone who has it can spend your credit. It never goes in an email, a chat message, a screenshot or a file you share.

Build it: make Scout's folder

1. Create the project folder and move into it:

```
```

2. Turn on Git for this folder:

```
```

3. Create a **virtual environment**, a private box for this project's Python packages so they never clash with anything else on your computer:

```
```

On Windows the second line is `.venv\Scripts\activate`. If PowerShell refuses with a message about scripts being disabled, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer Y, and try again. Your prompt now begins with `(.venv)`. **Every time you come back to Scout, run `cd scout` and the activate line first.** Most "it worked yesterday" problems are a missing `(.venv)`.

4. Store your API key. Open the `scout` folder in your text editor, create a new file called exactly `.env` (with the leading dot), and put one line in it, using your own key:

```
```

Save it. You wrote this file by hand on purpose: in Chapter 6 you will forbid Claude Code from ever reading it.

Build it: your first conversation with Claude Code

With `(.venv)` showing and the terminal inside the `scout` folder, type:

```
claude
```

The first time, it asks you to sign in through your browser, and then whether you trust the files in this folder. Say yes; it is your folder.

You are now talking to an agent that can act on your computer, within the folder you started it in. Try this:

```
I'm new to programming. Look around this folder and tell me, in
plain English,
what is here and what each item is for. Don't open .env.
```

Watch what happens. Claude does not answer straight away. It runs a command to list the files, perhaps reads one or two, and only then replies. That pause is the agent loop from Chapter 1, happening in front of you.

When Claude wants to do something that changes your computer, such as writing a file or running a command, it stops and asks. You will see the exact action and a choice to allow it once, allow it for the rest of the session, or say no. Read these prompts. They are the main way you stay in charge.

A few controls are worth learning now:

Key or command	What it does
Esc	Stops Claude mid-action so you can redirect it
Esc twice	Rewinds to an earlier point in the conversation, undoing Claude's file changes since then
Shift + Tab	Cycles the permission mode: normal (asks before edits), accept edits (stops asking about file changes), and plan (looks but changes nothing)
`/clear`	Starts a fresh conversation in the same folder; use it between unrelated tasks
`/help`	Lists every command
`/exit`	Leaves Claude Code

To finish the chapter, ask Claude to commit your empty project, so Git has a starting point:

```
Create a .gitignore that ignores .env, .venv, __pycache__ and
scout.db,
then make a first git commit with the message "Start Scout".
```

Checkpoint. `git log --oneline` shows one commit. `git status` does not mention `.env`. If it does, your `.gitignore` is wrong; ask Claude to fix it before going further.

Check your understanding

1. Why does Scout need its own API key when Claude Code already works through your subscription?
2. What does `(.venv)` at the start of your terminal prompt tell you, and what do you type if it is missing?
3. What is the difference between pressing Esc once and pressing it twice in Claude Code?
4. Name the three permission modes you can cycle through with Shift + Tab, and say when you would use plan mode.
5. Why did you create `.env` by hand rather than asking Claude Code to do it?

Exercise. Ask Claude Code: *"Explain what a git commit is, using an analogy, in under 100 words. Then show me the history of this project."* Notice which commands it asks permission to run, and which it runs without asking.

Sources: Install Claude Code; Claude Code 101.

Chapter 3. Talking to Claude well

A prompt is a briefing for a very capable new colleague who knows nothing about your situation. Almost every prompt-writing technique follows from taking that sentence seriously.

Anthropic's prompt engineering guide offers a simple test: show your prompt to a friend who knows nothing about the task, and ask whether they could do it well from what you wrote. If they would need to ask questions, Claude needs the answers too. This chapter turns that idea into a handful of habits, then uses them to write the first piece of Scout: the planner's prompt.

Seven habits

1. **Be clear and direct.** Say what you want, who it is for and what a good result looks like. "Summarise this" leaves a dozen decisions to chance. "Summarise this for a busy finance director in five bullet points, leading with the cost" leaves none.
2. **Give the reason.** Claude follows rules better when it knows why they exist. "Keep it under 600 words because readers see it on a phone" works better than "under 600 words".
3. **Set the role in a system prompt.** The **system prompt** is a separate briefing that sits above the conversation and frames everything in it. "You are the fact-checker for a research assistant" changes how every later message is read.
4. **Separate the parts with tags.** When a prompt contains several kinds of material, such as instructions, a question and some documents, wrap each in XML-style tags: `<question>...`

`</question>`, `<findings>...</findings>`. Claude can then refer to each part precisely and never mistakes your material for your instructions.

5. **Show examples.** One or two examples of the output you want are worth a paragraph of description. Make them varied, or Claude will copy their quirks.
6. **Make room to think.** For a hard decision, ask Claude to work out its approach before committing. Newer Claude models can also do this in a separate thinking step that the API can switch on.
7. **Say what to do, not only what to avoid.** "Write in flowing paragraphs" works better than "don't use bullet points".

One more principle comes from Anthropic's multi-agent article, and it matters most for agents: **teach heuristics, not scripts.** You cannot predict every situation an agent will meet. Give it the judgement a good human would use, such as "start with broad searches, then narrow down" or "prefer primary sources", rather than a rigid list of steps.

How much effort?

The same article describes an early failure of their research system: it would launch fifty subagents for a simple question. Their fix was to write effort rules straight into the lead agent's prompt: one agent for a simple fact, a few for a comparison, more for a broad investigation. Scout's planner does the same, with a hard ceiling of five, because each researcher costs money.

Build it: install Scout's packages

Start Claude Code in your `scout` folder (with `(.venv)` showing) and type:

```
Create a requirements.txt with these packages: anthropic≥2.0,
langgraph≥1.0,
fastapi≥0.115, uvicorn≥0.30, python-dotenv≥1.0, mcp≥2.0,
pytest≥8.0,
httpx≥0.27. Then install them into the active virtual
environment with pip.
```

Claude will ask before running `pip install`. Allow it. This takes a minute or two.

Next, the two small files every other part of Scout will use:

```
Create a Python package folder called scout with an empty
__init__.py, plus:
1. scout/config.py, which loads .env from the project root with
python-dotenv and
defines LEAD_MODEL (env SCOUT_LEAD_MODEL, default claude-sonnet-
5-5), WORKER_MODEL
(env SCOUT_WORKER_MODEL, default claude-haiku-5-5), and these
limits:
MAX_SUBTASKS=5, MAX_RESEARCH_TURNS=8,
MAX_SEARCHES_PER_RESEARCHER=5,
MAX_FINDINGS_PER_RESEARCHER=8, MAX_REVISIONS=1.
2. scout/llm.py, which keeps one shared anthropic.Anthropic()
client, with a
client() function to get it and a set_client() function so tests
can swap in a fake.
3. scout/prompts.py with load_prompt(name) that reads
scout/prompts/<name>.md.
Add short comments a beginner could follow. Do not read .env.
```

Notice how this prompt follows the habits above: it is specific about names and values, it says who the code is for, and it states the one thing not to do. Compare Claude's files with `scout/config.py`, `scout/llm.py` and `scout/prompts.py` in Appendix D.

Two models appear here. The **lead model**, Sonnet, plans, writes and reviews: the jobs that need judgement. The **worker model**, Haiku, does the searching: many calls, each simple, where speed and price matter more. Anthropic's own research system uses the same split of a stronger lead and cheaper workers.

Build it: write the planner's prompt yourself

This one you write by hand, because writing prompts is the skill. Create the folder `scout/prompts` and in it a file called `planner.md`. Type in the prompt below, or your own version of it. Read it as you go and notice each habit at work.

```
You are the lead researcher for Scout, a research assistant. You
do not search the
web yourself. Your job is to read the user's question and split
it into a small
number of independent research tasks, each of which will be
handed to a separate
researcher who works alone and cannot see the other tasks.

How to decide how many tasks to create:
- A simple factual question (one fact, one number, one date)
needs exactly 1 task.
- A comparison of two to four things needs one task per thing, or
2 to 4 tasks in total.
- A broad question with several distinct parts needs 3 to 5
tasks.
- Never create more than 5 tasks. Fewer, well-defined tasks beat
many vague ones.

Every task must stand on its own. A researcher reading only that
task should know:
1. The objective: exactly what to find out, in one or two
sentences.
2. The boundaries: what is out of scope, so two researchers never
search for the same thing.
3. Search hints: two or three short starting search phrases (two
to five words each).
Start broad; the researcher will narrow down.

If the question depends on recent events, say so in the objective
so the researcher
looks for recent sources.

Call the submit_plan tool with your tasks. Do not answer the
question yourself.
```

The middle section comes straight from Anthropic's lesson on delegation: a researcher given only "research the semiconductor shortage" will duplicate its colleagues' work or wander off topic. Objective, boundaries and starting points fix that. The last line mentions a tool you have not built yet; it arrives in Chapter 7.

Build it: try the prompt

Ask Claude Code for a throwaway script:

```
Create scratch/try_planner.py. It takes a question from the
command line, sends it to
config.LEAD_MODEL with the planner prompt as the system prompt
and the question wrapped
in <question> tags, but instead of a tool it adds this to the end
of the question:
"Write your tasks as a numbered list inside <plan> tags." Print
Claude's reply.
```

Then run it with a few different questions:

```
python3 scratch/try_planner.py "What year did the Berlin Wall
fall?"
python3 scratch/try_planner.py "Compare the running costs of heat
pumps and gas boilers in the UK"
```

The first should come back with one task; the second with two to four. If your planner creates five tasks for the Berlin Wall, its effort rules are not landing. Edit `planner.md`, save it and run again. No code changes are needed: that is why the prompts live in their own files.

Each run costs a fraction of a penny. Look at your usage page on platform.claude.com after a few runs, so you get a feel for what things cost.

Checkpoint. You have `requirements.txt`, `scout/config.py`, `scout/llm.py`, `scout/prompts.py` and `scout/prompts/planner.md`, and the trial script produces sensible plans. Ask Claude to commit with the message "Planner prompt".

Check your understanding

1. What is the "friend test" for a prompt?
2. Why does Scout's planner prompt say how many tasks to create for different kinds of question?
3. What are XML-style tags for in a prompt? Give an example from this chapter.
4. Why does Scout use one model for planning and writing and a different one for searching?
5. Rewrite this weak prompt using at least three habits from the chapter: *"Tell me about electric cars."*

Exercise. Add one example plan to the end of `planner.md`, for a question unlike the ones you will test with. Run the trial script on three questions before and after. Did the example help, or did Claude start copying it too closely?

Sources: Prompt engineering overview; How we built our multi-agent research system.

Part Two: Working with Claude Code

Chapter 4. Building with Claude Code

The most reliable way to build with a coding agent is a four-beat rhythm: explore, plan, build, check. Skip a beat and you will spend longer fixing than you saved.

- **Explore.** Let Claude read the relevant files before it writes anything. An agent that has not looked will guess, and guesses compound.
- **Plan.** Ask for a plan and read it before any file changes. A wrong turn in a plan costs one sentence to fix; in code it can cost an hour.
- **Build.** Let Claude make the change, in a step small enough that you can check it.
- **Check.** Run it, look at it, and commit it to Git when it works. A commit is a save point you can always return to.

Claude Code has a mode made for the second beat. Press **Shift + Tab** until the bottom of the screen says *plan mode*. In plan mode Claude can read and search but cannot change anything. It explores, then presents a plan and asks whether to go ahead. You can approve it, or reply with corrections and get a revised plan.

Giving Claude the right context

Claude Code only knows what is in its context window. A few ways to put the right things there:

- **Point at files with @.** Typing `@scout/config.py` in your message pulls that file in directly.
- **Paste pictures.** If the web page looks wrong, take a screenshot and paste or drag it into Claude Code. It can see images, and "make it look less cramped, like this" plus a picture is clearer than

any description.

- **Paste error messages whole.** Do not paraphrase an error. Copy every line.
- **Clear between tasks.** `/clear` starts a fresh conversation. Old, unrelated material in the context makes Claude worse, not better.

What Scout's web page needs

Before any agent exists, Scout needs a body: a page to type a question into, a server to receive it, and somewhere to keep the answers. The pieces are:

File	Job
`scout/store.py`	Saves every question, its progress messages and its final report in a small database file, `scout.db`
`scout/server.py`	The **web server**: a program that waits for the browser to ask for things and answers
`scout/static/index.html`	The single web page you see
`scout/graph.py`	Will hold the research workflow. For now, a stand-in that returns a fake report

Research takes minutes, but a web page should not freeze for minutes. So the page sends the question, gets back a job number immediately, and then checks in every two seconds to see how the job is going. The research runs in the background on the server. This is a common pattern and worth recognising: start a job, then poll it.

Build it: plan the skeleton

Start Claude Code, switch to plan mode with Shift + Tab, and type:

```
I want to build the web part of Scout, a research assistant. No
AI yet: a stand-in
function will fake the research. Please plan:

1. scout/store.py: SQLite storage of research jobs (id, question,
status running/done/
failed, a list of progress messages, report, error, created_at).
Functions:
create_job, add_progress, finish_job, fail_job, get_job,
list_jobs, count_jobs_today.
Database path comes from config.DB_PATH (add it to config.py,
default scout.db in
the project root).
2. scout/graph.py: for now, run_research(question,
on_progress=None) that calls
on_progress with three fake messages, one second apart, and
returns
{"report": "# Placeholder\nThis is where the real answer will
go."}.
3. scout/server.py: a FastAPI app. GET / serves
scout/static/index.html.
POST /api/research takes {"question": ...}, rejects questions
under 10 or over 500
characters, creates a job, runs run_research in a background task
and returns
the job id. GET /api/research/{id} returns the job. GET
/api/history returns recent
jobs. GET /health returns {"ok": true}.
4. scout/static/index.html: one calm, readable page with a
question box and button.
It posts the question, then polls the job every 2 seconds, shows
progress messages
as a list and renders the finished Markdown report using marked
and DOMPurify from
cdnjs. Show recent questions underneath. Support dark mode.

Explain the plan in plain English before any code.
```

Read the plan Claude gives you. You are not checking the code, which does not exist yet; you are checking that it understood. Does it mention every file? Does it agree that the research happens in the background? If something is off, say so in plain words and ask for a revised plan. When it looks right, approve it and let Claude build.

You will see a stream of permission prompts as it creates files. In this project, accepting file edits for the session is reasonable; keep reading the commands it wants to run.

Build it: see it work

When Claude says it is done, start the server yourself in a second terminal window, so you can keep talking to Claude in the first:

```
cd scout
source .venv/bin/activate
uvicorn scout.server:app --reload
```

The last line it prints says `Uvicorn running on http://127.0.0.1:8000`. Open that address in your browser. You should see Scout's page. Type a question of at least ten characters and press the button. Three progress lines should appear over about three seconds, then the placeholder report.

The `--reload` option makes the server restart whenever a file changes, so you can leave it running while you work. Stop it with Ctrl + C.

If something is wrong, tell Claude what you see, not what you think the cause is. "I pressed the button and nothing happened" is useful. Better still: in the browser, right-click the page, choose *Inspect*, open the *Console* tab, and paste any red error lines to Claude.

If Claude makes a change you do not like, press **Esc twice** to open the rewind menu and pick a point before it. Claude Code keeps checkpoints of its own file edits, so you can undo a bad direction without touching Git. Git is still your long-term safety net; checkpoints are for the next five minutes.

Checkpoint. The page works end to end with the fake report, and your recent questions appear under the box after a page refresh. Compare your files with Appendix D; the real `graph.py` there is the Chapter 9 version, so expect that one to differ. Then:

```
Commit this with the message "Web skeleton with stand-in
research".
```

Check your understanding

1. What are the four beats of the rhythm in this chapter, and which one does plan mode support?
2. Why does the page get a job number back immediately instead of waiting for the report?
3. You want Claude to look at `scout/server.py` specifically. How do you point it there?
4. What is the difference between rewinding with Esc twice and going back to a Git commit?
5. The page is blank after you click the button. What two things should you give Claude to help it find the problem?

Exercise. Without looking at the code, ask Claude in plan mode: *"Plan how to add a 'clear' button that empties the question box. Do not make changes."* Read the plan, then reject it with a change of your own (for example, the button should also clear the report). Notice how quickly a plan changes compared with code.

Sources: Claude Code in action; Claude Code 101.

Chapter 5. Giving Claude a memory

Every Claude Code session starts with an empty head. Close the terminal, open it tomorrow, and Claude has no idea what Scout is, how to run it, or that you asked it twice not to touch `.env`. Claude Code has two ways to carry knowledge from one session to the next: **CLAUDE.md files**, which you write, and **auto memory**, which Claude writes for itself.

CLAUDE.md: the project's briefing note

A `CLAUDE.md` file is plain text that Claude Code reads at the start of every session in that folder. Think of it as the note you would leave for a contractor who arrives on Monday morning: how the building works, where the tools are, and what not to do.

There can be several, at different levels, and Claude reads all of them:

File	Who it applies to	Good for
`~/.claude/CLAUDE.md`	You, in every project	"Explain things in plain English"; "I use Windows"
`CLAUDE.md` in the project folder	Everyone working on this project	Commands, layout, project rules. Committed to Git
`CLAUDE.local.md` in the project folder	You, in this project only	Personal notes you do not want to share. Not committed
`.claude/rules/*.md`	Everyone, optionally only for certain files	Rules that matter for one part of the project

The ~ means your home folder.

What belongs in it

Anthropic's documentation gives a sensible test: add something to CLAUDE.md when Claude makes the same mistake twice, when you find yourself typing the same correction you typed last session, or when a new teammate would need to be told. In practice that means:

- **Commands**: how to set up, run, test.
- **Layout**: which file does what.
- **Rules**: what must always or never happen.

It does not mean pasting in everything you know. The documentation recommends keeping each file under about 200 lines, because a long file takes up context and makes Claude less likely to follow any single instruction in it. Be concrete: "Run `python -m pytest -q` before committing" beats "make sure things work". And keep it consistent; two rules that contradict each other leave Claude to pick one.

Two more features are worth knowing. A line such as `@docs/architecture.md` inside CLAUDE.md pulls that whole file in, which keeps the main file short. And a rule file in `.claude/rules/` can start with a `paths:` list so it only loads when Claude is working on matching files:

```
---
paths:
- "scout/prompts/**"
---
Prompts are read by an AI, not a person. Keep them in plain
English, explain the
reason behind each rule, and never put a real API key or URL with
a token in them.
```

Auto memory: Claude's own notebook

Auto memory is a folder of notes that Claude keeps for itself, one per project, inside `~/.claude/projects/`. When you correct Claude ("no, on my machine it's `python`, not `python3`"), it may decide that is worth

remembering and write it down. It reads the first 200 lines of its index file, `MEMORY.md`, at the start of each session.

You do not have to manage it, but you should know it is there. Type `/memory` in Claude Code to see every CLAUDE.md file that is loaded, switch auto memory on or off, and open the folder of notes. Everything in it is plain text you can read, edit or delete.

One caution applies to both kinds of memory. They are **advice, not law**. Claude reads them and tries to follow them, but nothing forces it to. If something must happen every time, or must never happen, you need the next chapter.

Build it: Scout's CLAUDE.md

1. In Claude Code, type:

```
```

Claude explores the project and writes a first draft of `CLAUDE.md`. Read it. It will have guessed some things well and missed the rules only you know.

2. Now shape it. Ask:

```
```

Compare the result with the version in Appendix D. Some of the layout lines there describe files you have not built yet. That is fine; update them as you go.

3. Test it. Type `/clear` to wipe the conversation, then ask:

```
```

Claude should answer correctly without exploring, because the answer is in the note it read at startup.

4. Tell Claude something about yourself and see where it goes:

```
```

Then open `/memory` and look at the auto memory folder. You will find a new note.

Checkpoint. `CLAUDE.md` exists in the project folder, is under 200 lines, and a fresh session knows how to run Scout. Commit it with the message "Project notes for Claude".

Check your understanding

1. What is the difference between CLAUDE.md and auto memory?
2. You want a rule that applies to you in every project. Where does it go?
3. Why is "under about 200 lines" good advice for a CLAUDE.md file?
4. What does a `paths:` list at the top of a `.claude/rules/` file do?
5. CLAUDE.md says "never edit .env". Can you rely on that? Why or why not?

Exercise. Create `.claude/rules/prompts.md` with the example rule from this chapter. Start a new session and ask Claude to suggest one improvement to `scout/prompts/planner.md`. Then run `/memory` and check whether the rule was loaded.

Source: How Claude remembers your project.

Chapter 6. Checks that can say no

A hook is a small program that Claude Code runs automatically at a fixed moment, such as just before Claude edits a file or just before it says it has finished. Unlike CLAUDE.md, a hook is not advice. It runs every time, and some hooks can stop Claude in its tracks.

This is the chapter where you make Scout's project safe to hand to an agent.

The moments you can hook into

Claude Code has many hook events. Four do most of the useful work:

Event	When it runs	Can it say no?	Typical use
`SessionStart`	When a session begins	No	Add up-to-date context, such as today's open issues
`PreToolUse`	Just before Claude uses a tool: edit a file, run a command	**Yes**, it can block the action	Protect secret files; block dangerous commands
`PostToolUse`	Just after a tool succeeds	No, but it can tell Claude something	Tidy the formatting of a file Claude just wrote
`Stop`	When Claude is about to finish its turn	**Yes**, it can send Claude back to work	Refuse to let Claude stop while tests are failing

How a hook talks to Claude Code

When an event fires, Claude Code starts your hook program and hands it a description of what is happening, as **JSON** (a simple text format of names and values) on its input. For a `PreToolUse` hook before a command, that looks like this, trimmed:

```
{ "hook_event_name": "PreToolUse",
"tool_name": "Bash",
"tool_input": { "command": "rm -rf build" } }
```

The hook answers in one of two ways.

- **With its exit code**, the number every program returns when it ends. Zero means "no objection". **Two** means "block this", and whatever the hook printed to its error output is shown to Claude as the reason. Any other number counts as a hook that broke, and the action goes ahead. That catches people out: use 2, not 1, when you mean no.
- **With a JSON reply**, which allows finer control. For `PreToolUse`, a reply with `"permissionDecision": "deny"` and a reason blocks the action and tells Claude why.

A hook that says nothing does not approve anything; the normal permission prompts still apply. And a `PreToolUse` deny wins even if you have told Claude Code to stop asking permission. That makes hooks the right place for rules that must never be broken.

Hooks are set up in settings files. `.claude/settings.json` in the project is shared through Git with everyone who works on it; `.claude/settings.local.json` is just for you; `~/.claude/settings.json` applies to all your projects. Type `/hooks` in Claude Code to see every hook that is active and where it came from.

A warning that cuts both ways: hooks run with your full permissions on your computer. That is what makes them powerful, and it is why you should read any hook that comes in someone else's project before you trust that folder.

Build it: tests first

Your Stop hook will refuse to let Claude finish while tests fail, so there must be tests. Ask Claude Code:

```
Create tests/__init__.py and tests/test_server.py using FastAPI's
TestClient.
Test that: /health returns ok; a question under 10 characters is
rejected with 400;
a good question returns an id and the job then finishes with a
report; /api/history
lists it. Use a temporary database for each test by setting
config.DB_PATH in a
pytest fixture in tests/conftest.py. Then run python -m pytest -
q.
```

You should see a line ending in `passed`. A **test** is a small program that checks another program does what it should. Tests are how an agent, and you, can tell that a change broke nothing.

Build it: the guard

Create the folder `.claude/hooks` and ask Claude to create `.claude/hooks/protect.py` by pasting the listing from Appendix D after the words "Create this file with exactly this content:". (Claude is allowed to; the guard is not switched on yet.) Here is what it does, in plain terms:

- If Claude tries to **edit** a file named `.env`, block it: secrets are for humans.
- If Claude tries to edit `evals/questions.jsonl` or `evals/run_eval.py`, block it: in Chapter 14 an agent will be graded by those files, and nobody gets to mark their own exam.
- If Claude tries to **run** a command containing `rm -rf` or a forced Git push, block it.
- If a command would print `.env` to the screen with `cat`, `head` and so on, block it.
- Otherwise, say nothing.

The heart of it is the deny reply:

```
def deny(reason):
print(json.dumps({"hookSpecificOutput": {
"hookEventName": "PreToolUse",
"permissionDecision": "deny",
"permissionDecisionReason": reason}}))
sys.exit(0)
```

Build it: the finishing check

Ask Claude to create `.claude/hooks/run_tests.py` as in Appendix D. When Claude is about to stop, it runs the tests. If they pass, it exits 0 and Claude stops. If they fail, it prints the failures and exits **2**, which sends Claude back to fix them. It also checks a field called `stop_hook_active`: if Claude has already been sent back once this turn, the hook lets it stop, so the two can never get stuck in an endless loop.

Build it: switch them on

Create `.claude/settings.json` yourself in your editor, with this content:

```
{
"hooks": {
"PreToolUse": [
{ "matcher": "Edit|Write|MultiEdit|Bash",
"hooks": [ { "type": "command", "command": "python3",
"args": ["${CLAUDE_PROJECT_DIR}/.claude/hooks/protect.py"] } ] }
],
"Stop": [
{ "hooks": [ { "type": "command", "command": "python3",
"args": ["${CLAUDE_PROJECT_DIR}/.claude/hooks/run_tests.py"],
"timeout": 300 } ] }
]
}
}
```

The `matcher` says which tools the guard watches. `${CLAUDE_PROJECT_DIR}` is filled in by Claude Code with your project folder, so the hook is found wherever Claude happens to be working. On Windows, change both `"python3"` to `"python"`. Always start Claude Code with `(.venv)` showing, so the hooks use the project's Python.

Restart Claude Code (`/exit`, then `claude`) and type `/hooks` to confirm both appear.

Build it: try to break the rules

Try each of these and watch Claude get refused and explain why:

```
Show me what's in .env.
Add a comment to the top of .env saying "secrets".
Delete the scratch folder using rm -rf scratch.
```

Now see the Stop hook at work. Ask Claude to break something on purpose:

```
Change the minimum question length in scout/server.py from 10 to
3, but don't
change the tests.
```

The test for short questions now fails. When Claude tries to finish, the hook sends it back with the failure, and it will either fix the code or ask you which behaviour you want. Answer it, then ask it to put the limit back to 10.

Checkpoint. All three rule-breaking requests are refused; tests pass; `/hooks` lists both hooks. Commit with "Safety hooks".

Check your understanding

1. What can a hook do that a line in CLAUDE.md cannot?
2. A `PreToolUse` hook exits with code 1. Is the action blocked? What should it have used?

3. Why does the Stop hook check `stop_hook_active`?
4. Which settings file would you put a hook in so everyone working on the project gets it?
5. Why is the guard protecting the evaluation files, even though they do not exist yet?

Exercise. Add a `PostToolUse` hook, matching `Edit|Write`, that appends the time and the edited file's path to a file called `edits.log`. (Hint: ask Claude to write it, and to read `tool_input.file_path` from the JSON.) Make a small edit and check the log.

Sources: Hooks reference.

Part Three: Building the agent

Chapter 7. Writing tools for agents

A tool is a promise between two very different parties: ordinary code, which does exactly the same thing every time, and a model, which might call it well, badly, or not at all. Anthropic's article *Writing effective tools for agents* argues that this calls for a different style of design from the one programmers use for each other, and that the effort pays off more than almost anything else you can do.

What a tool looks like to Claude

Claude never sees your tool's code. It sees three things: a **name**, a **description** in plain English, and an **input schema** listing what to supply, each with its own description. From those alone it decides whether to use the tool and how. So the description is not documentation for humans; it is part of the prompt.

Here is the one tool Scout's researchers will call that you write yourself:

```
RECORD_FINDING_TOOL = {
"name": "record_finding",
"description": (
"Save one fact you found during research so it can be used in the
final report. "
"Call this once per fact, as soon as you find it. Only recorded
findings reach the "
"report; anything not recorded is lost. Do not record opinions or
guesses, and do "
"not record the same fact twice."),
"input_schema": {
"type": "object",
"properties": {
"claim": {"type": "string", "description": "The fact, written as
one complete "
"sentence in your own words... Example: 'The Golden Gate Bridge
opened "
"to traffic on 28 May 1937.'"},
"source_url": {"type": "string", "description": "The full web
address (starting "
"with https://) of the page that states this fact."},
"source_title": {"type": "string", "description": "The title of
that page..."}},
"required": ["claim", "source_url", "source_title"]}}
```

Read the description as Claude would. It says what the tool does, when to call it, why it matters ("anything not recorded is lost") and what not to do. The inputs have unambiguous names and an example. The article's advice is to write descriptions as you would explain the tool to a new hire.

Five principles from the article

1. **Build fewer, better tools.** Do not wrap every function you have. An agent given `list_contacts` will read thousands of names one by one, wasting its limited context. Give it `search_contacts` instead. Combine steps that always go together into one tool.
2. **Name tools so they cannot be confused.** When an agent has dozens of tools, grouping names with a shared prefix (`asana_search`, `jira_search`) helps it pick the right one.

3. **Return what is meaningful.** Give back names and readable text, not internal codes. The article reports that replacing random-looking IDs with plain words noticeably cut down on mistakes.
4. **Be economical.** Every word a tool returns costs context. Limit, filter and paginate, and when you cut something short, say so and say how to get more.
5. **Make errors teach.** "Error 422" teaches nothing. "source_url must start with https://, for example https://www.example.org/page; you sent 'example.org'" lets the agent fix itself on the next try.

Scout's `handle_record_finding` function, the code that runs when Claude calls the tool, follows the last two: every rejection says what was wrong and what to do, and the success message tells the agent how many more findings it may record, which nudges it to stop at a sensible point.

Two kinds of tool

Scout's researchers use two tools, and they work differently.

- `web_search` is a **server tool**. It runs on Anthropic's servers. You only switch it on, with a limit on searches; Claude performs the search and reads the results within a single request.
- `record_finding` is a **client tool**. When Claude wants it, the response stops with `stop_reason` set to `"tool_use"`. Your code runs the function and sends the result back in the next message.

There is a third use for tools, which the planner relies on: **asking for a structured answer.** Offer Claude a single tool and ask it, in the prompt, to answer by calling that tool. The planner must call `submit_plan` with a list of tasks, so your code receives tidy data instead of prose it has to pick apart. Some models also let you force this with a setting called `tool_choice`, but the current Sonnet model

rejects it, so Scout's `ask_for_tool` helper checks the reply instead and reminds Claude once if it answered in prose. This replaces the `<plan>` tags you used in Chapter 3.

The agent loop in code

This is the heart of every agent you will ever build. Here it is from `scout/agents.py`, slightly simplified:

```
for turn in range(config.MAX_RESEARCH_TURNS): # never loop
forever
response = llm.client().messages.create(
model=config.WORKER_MODEL, system=load_prompt("researcher"),
tools=[WEB_SEARCH_TOOL, RECORD_FINDING_TOOL], messages=messages,
max_tokens=4000)
messages.append({"role": "assistant", "content":
response.content})

if response.stop_reason == "tool_use": # Claude wants our tool
results = []
for block in response.content:
if block.type == "tool_use" and block.name == "record_finding":
output = handle_record_finding(block.input, findings,
subtask["title"])
results.append({"type": "tool_result", "tool_use_id": block.id,
"content": output})
messages.append({"role": "user", "content": results})
continue # go round again
if response.stop_reason == "pause_turn": # a long search paused
continue # send it back to carry on
break # Claude has finished
```

Think, act, observe, repeat, with a hard limit on turns. Everything else in this book is built around loops like this one.

Build it

1. Write the researcher's prompt yourself, as you did the planner's. Create `scout/prompts/researcher.md` and copy the version from Appendix D, reading it as you go. Notice the search advice (short

and broad first), the source-quality advice and the stopping rule. Each answers a failure Anthropic saw in its own research agents.

2. Ask Claude Code, in plan mode:

```
```

3. Read the plan, approve it, and let Claude build. Your Stop hook will make sure the tests pass before it finishes.
4. Now a real test, which costs a penny or two. Ask for a throwaway script:

```
```

Run it:

```
```

After twenty seconds or so you should see two to five findings, each with a real web address. Open one or two of the addresses and check that the page really says what the claim says. You are now doing what Anthropic's article calls evaluation, by hand: watching how an agent uses your tools and noticing where it stumbles.

If you see an error about web search not being enabled, return to the Console settings from Chapter 2.

Checkpoint. Tests pass; the trial script prints sourced findings. Compare `scout/tools.py` and `scout/agents.py` with Appendix D (the writer and reviewer in that file arrive next chapter). Commit with "Tools and researcher agent".

Check your understanding

1. Claude cannot see your tool's code. What three things does it see?
2. Why is `search_contacts` a better tool for an agent than `list_contacts`?

3. What is the difference between a server tool and a client tool? Give one example of each from Scout.
4. How does Scout make sure the planner returns a list of tasks rather than a paragraph of prose?
5. Why does the agent loop have a `range(config.MAX_RESEARCH_TURNS)` around it?

Exercise. Make `handle_record_finding`'s error messages worse on purpose: change every rejection to just `"Error"`. Run the trial script on a tricky question a few times and watch whether the researcher recovers from mistakes. Then put the helpful messages back.

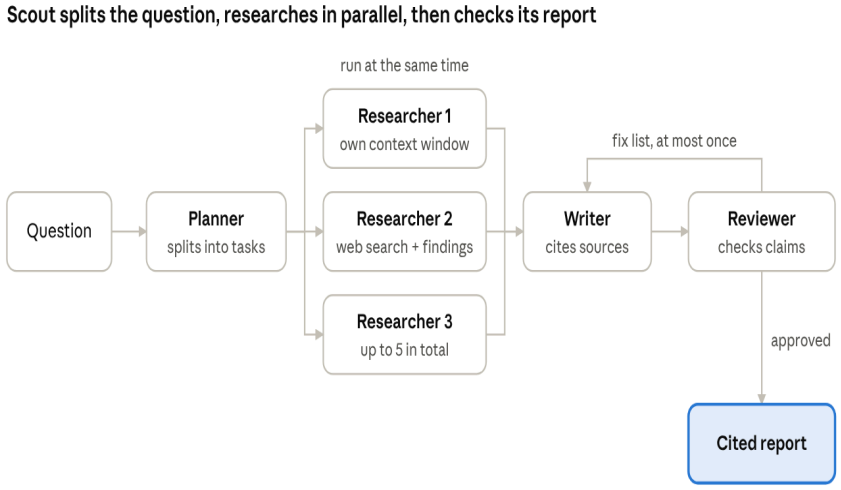
Sources: Writing effective tools for agents; Web search tool.

Chapter 8. A team of researchers

In this chapter Scout answers its first real question. The planner splits it up, several researchers search at the same time, a writer turns their findings into a report with numbered sources, and a reviewer checks the report before you see it. The design is a small copy of the one Anthropic describes in *How we built our multi-agent research system*.

Why a team works

The article's key observation is that searching is mostly **compression**: reading a great deal and keeping only what matters. Each researcher reads widely in its own fresh context window, then hands back a few short findings. The lead never has to hold every web page in its head, only the distilled results. Splitting the work also keeps each researcher focused on one thread, so an early wrong turn in one does not drag the others down.



Scout's research team · orchestrator, parallel workers, evaluator

The planner is the orchestrator, the researchers are parallel workers, and the reviewer and writer form an evaluator-optimizer pair: three of Chapter 1's patterns in one system.

The article is just as frank about the costs and failure modes, and Scout is designed around each one:

Lesson from the article	What Scout does about it
Vague delegation causes duplicated or missing work	Every task has an objective, boundaries and search hints (Chapter 3)
Agents overspend on simple questions	Effort rules in the planner, and hard limits in `config.py`
Long, specific searches find little	The researcher prompt says start short and broad, then narrow
Agents favour search-engine-optimised content farms	The researcher prompt asks for primary sources
Doing things one after another is slow	Researchers run in parallel
Claims need checking against sources	Every claim carries a source; a reviewer checks citations
Multi-agent systems use roughly 15 times the tokens of a chat	Cheap worker model, capped tasks, searches and findings

How the citations work

Anthropic's system ends with a separate citation agent that matches each claim to its source. Scout takes a simpler route that is very hard to get wrong. Every finding already carries its web address. Before the writer starts, ordinary code numbers each distinct web page: 1, 2, 3. The writer sees findings labelled with those numbers and must cite them, like this: "The telescope launched on 25 December 2021 [3]." After it finishes, code, not the model, appends the numbered list of sources. The model never types a web address, so it cannot invent one.

The reviewer then has two layers. A plain function, `check_citations`, catches mechanical errors such as a citation to source 9 when there are only seven. Then the reviewer model reads the draft against the

findings, the way Anthropic's grading model checked factual and citation accuracy, and either approves it or returns a list of specific fixes. If it rejects the draft, the writer tries once more with the fixes in hand. That is the evaluator-optimizer pattern from Chapter 1.

Running researchers in parallel

Waiting for one researcher before starting the next would make Scout three or four times slower. Python can run several at once with a **thread pool**: a small team of workers inside the program, each handed one task. Each researcher spends most of its time waiting for Claude to reply, so running them side by side costs almost nothing extra. In the next chapter you will replace this hand-built version with LangGraph, which does the same thing more tidily.

Build it

1. Create `scout/prompts/writer.md` and `scout/prompts/reviewer.md` from Appendix D. Read both. Notice that the writer is told to use only the findings, to lead with the answer, to cite every factual sentence and to admit gaps; and that the reviewer is told that small matters of style are not a reason to reject.
2. Ask Claude Code, in plan mode:

```
```

3. Approve and let it build. The Stop hook will hold it to passing tests.

Build it: the first real question

Start the server (`uvicorn scout.server:app --reload`) if it is not running, open the page and ask:

```
How does the James Webb Space Telescope differ from the Hubble
Space Telescope?
```

Watch the progress list. You should see a plan with two to four tasks, a line for each researcher as it finishes (often not in the order they were listed: they really are running at once), a draft, and the fact-checker's verdict. After one to three minutes the report appears, with numbered citations and a list of sources at the bottom.

Now do what Anthropic's team did and read it as a sceptical human. Click three of the sources. Does each page say what the sentence citing it says? Is any source a content farm? Is there anything in the report you know to be wrong? Write down what you find. Those notes are the start of the evaluation you will build in Chapter 10.

Check the cost on your usage page. A question like this typically costs somewhere between a few pence and twenty pence, depending mostly on how many searches the researchers ran.

Checkpoint. A real question produces a cited report in the browser, and tests pass. Commit with "Research team: planner, researchers, writer, reviewer".

Check your understanding

1. In what sense is searching "compression", and why does that favour giving each researcher its own context window?
2. Why does Scout build the list of sources in code instead of letting the writer type the web addresses?
3. What two layers of checking does the reviewer apply?
4. Why does running researchers in parallel cost almost nothing extra in waiting time?
5. Name two failure modes from Anthropic's article and the part of Scout that guards against each.

Exercise. Ask Scout a simple question, such as "What is the capital of Australia?" Check the progress list: how many tasks did the planner make? Then ask a broad question with several parts. If the number of

tasks does not scale sensibly, adjust the effort rules in `planner.md` and try again.

Source: How we built our multi-agent research system.

Chapter 9. Drawing the map

LangGraph is a library for describing an agent system as a map: boxes for the steps, arrows for what happens next, and a shared notebook that every step can read and add to. In this chapter you redraw Scout's workflow as such a map. It will do exactly what it did before, and that is the point: you will see the same machine with its structure made visible.

You may wonder why the book had you build the hand-made version first. *Building effective agents* warns that frameworks can hide what is really happening underneath, making problems harder to find, and recommends understanding the plain version before adopting one. You now know what LangGraph will be doing for you, because you did it yourself in Chapter 8.

Four ideas

- **State** is the shared notebook. For Scout it holds the question, the plan, the findings, the report, the review and a count of revisions.
- **Nodes** are the steps. Each is an ordinary function that reads the state and returns the parts it wants to change. Scout has four: `plan`, `research`, `write` and `review`.
- **Edges** are the arrows. A normal edge always goes to the same next step. A **conditional edge** runs a small function that decides where to go: after `review`, either finish or go back to `write`.
- **Send** is how one step starts many. After `plan`, a function returns one `Send("research", task)` per task, and LangGraph runs all those research nodes at the same time, then waits for every one before moving on to `write`.

There is one subtlety. When several researchers finish at once, each returns its own list of findings. Which one wins? You tell LangGraph by marking the field with a **reducer**, a rule for combining updates. Scout marks `findings` with `operator.add`, which means "add each new list to the end of the old one". Without it, the researchers would overwrite each other and you would lose most of their work.

The graph in code

Here is the core of the new `scout/graph.py`. Read it slowly; it is short.

```
class ScoutState(TypedDict, total=False):
question: str
subtasks: list[dict]
findings: Annotated[list[dict], operator.add] # researchers ADD
to this list
report: str
sources: list[dict]
review: dict
revisions: int

def send_to_researchers(state):
return [Send("research", {"question": state["question"],
"subtask": t})
for t in state["subtasks"]]

def after_review(state):
if state["review"]["approved"] or state["revisions"] >
config.MAX_REVISIONS:
return END
return "write"

builder = StateGraph(ScoutState)
builder.add_node("plan", plan_node)
builder.add_node("research", research_node)
builder.add_node("write", write_node)
builder.add_node("review", review_node)
builder.add_edge(START, "plan")
builder.add_conditional_edges("plan", send_to_researchers,
["research"])
builder.add_edge("research", "write")
builder.add_edge("write", "review")
builder.add_conditional_edges("review", after_review, ["write",
END])
graph = builder.compile()
```

The node functions are one or two lines each, because they simply call the agents you already wrote. The `after_review` function holds the limit on rewrites; without it, a fussy reviewer and a stubborn writer could argue until your credit ran out.

The web page still needs progress messages. LangGraph's `stream` method reports each node's update as it finishes, so `run_research` loops over those updates, keeps its own copy of the state, and turns

each one into a short sentence for the page.

What else a graph gives you

Once the workflow is a graph, new possibilities open up cheaply. You can add a step by adding a node and two arrows. You can draw a picture of the whole system from the code itself. And LangGraph can attach a **checkpointer**, which saves the state after every step so a long run can pause for a human's approval or resume after a crash. Anthropic's article stresses this point: agents are long-running and errors compound, so being able to resume from where things went wrong, rather than starting again, matters in production. Scout's runs are short enough not to need it yet, but the door is open.

Build it

1. Ask Claude Code, in plan mode:

The last line matters. The tests describe what Scout should do. If Claude can rewrite the inside without changing them and they still pass, you have good evidence that nothing broke.

2. When it is done, draw the map:

Claude prints a short text description of a diagram in a format called Mermaid. Copy it, open mermaid.live in your browser, and paste it into the editor. You will see Scout's workflow drawn as boxes and arrows, generated from your own code. Dotted arrows are the conditional ones.

3. Ask Scout a real question in the browser to confirm nothing changed from the outside.

Checkpoint. Tests pass without being edited; a live question still works; your drawing shows plan, research, write and review with a loop between the last two. Compare `scout/graph.py` with Appendix D and commit with "Workflow as a LangGraph graph".

Check your understanding

1. What are state, nodes and edges in LangGraph? Give Scout's example of each.
2. What does `Send` do, and where does Scout use it?
3. What would go wrong if `findings` had no reducer?
4. Why did you tell Claude not to change the tests during this rewrite?
5. What is a checkpointer for, and why does Anthropic's article care about resuming?

Exercise. Add a fifth node, `summarise`, between `review` and the end, which writes a single-sentence summary of the report to a new `summary` field in the state. Plan it with Claude first, draw the new map, and check the old tests still pass. Then decide whether it was worth it, remembering the principle of simplicity. You can always undo it with Git.

Sources: LangGraph documentation; Building effective agents; How we built our multi-agent research system.

Part Four: Scaling your workshop

Chapter 10. Helpers with their own desks

You have built a lead agent that hands work to helpers with their own fresh context. Claude Code can do exactly the same for you while you build. Its helpers are called **subagents**, and this chapter puts two of them to work: a code reviewer, and an examiner that runs Scout's new evaluation.

What a subagent is

A subagent is a separate Claude with its own context window, its own instructions and its own set of tools. The main Claude, the one you are talking to, hands it a task. The subagent does the work, which might mean reading thirty files or running a noisy command, and hands back only a summary. All the clutter stays on its desk, not yours.

The Claude Code documentation lists the benefits: keeping the main conversation clean, limiting what a helper is allowed to touch, reusing a well-briefed helper across projects, and running cheaper models for simple jobs. It is the same reasoning as Scout's researchers, applied to programming.

Claude Code comes with some built in. **Explore** is a fast, read-only searcher; **Plan** gathers information when you are in plan mode; **general-purpose** can do anything. Claude uses these on its own when it judges they would help. You can also ask for them directly, including several at once:

```
Using three separate Explore subagents in parallel, find: (1)
every place a limit
from config.py is used, (2) where each prompt file is loaded, (3)
where the
progress messages shown on the web page are created. Summarise
each in two lines.
```

Defining your own

A custom subagent is a Markdown file in `.claude/agents/` (for this project) or `~/.claude/agents/` (for every project). The top part, between the `---` lines, is its settings; the rest is its system prompt.

Setting	Meaning
`name`	Its identifier, lowercase with hyphens
`description`	When it should be used. Claude reads this to decide when to hand work over, so make it specific
`tools`	Which tools it may use. Leave a tool out and it simply cannot use it
`model`	`sonnet`, `opus`, `haiku`, or `inherit` to match the main conversation

There are more settings, including persistent memory for a subagent and running it in its own copy of the project, but these four take you a long way. Remember that a subagent starts with no knowledge of your conversation; its system prompt and the task Claude writes for it are all it has, so brief it as carefully as Scout's planner briefs a researcher.

You can ask for a subagent by name in plain words ("use the code-reviewer subagent"), or type `@` and choose it from the list to make sure it runs.

Build it: a reviewer

Create `.claude/agents/code-reviewer.md` from Appendix D. Its key lines are:

```
---
name: code-reviewer
description: Reviews recent code changes in Scout for bugs,
security problems and
unclear code. Use after any change to files in scout/ and before
committing.
tools: Read, Grep, Glob, Bash
model: sonnet
---
You are a careful senior reviewer looking at changes to Scout...
The person who owns
this project is not a programmer, so explain every finding in
plain English.
```

Its tool list leaves out Edit and Write, so it cannot change your files even by accident; its instructions tell it to report and nothing more. Its checklist puts secrets and cost risks first, because for an app that spends money every time it runs, those are the expensive mistakes.

Restart Claude Code, make a small change (ask Claude to add a docstring somewhere), then:

```
Use the code-reviewer subagent on my uncommitted changes.
```

Watch the transcript: a line appears showing the subagent's name and its task, and a little later a tidy report comes back, grouped as must fix, should fix and fine.

Build it: an exam for Scout

Anthropic's research team found two things about evaluating agents that apply directly to Scout. First, start small and start now: about twenty realistic questions were enough to see whether a change helped, because early improvements are large. Second, a model can grade free-text answers reliably if you give it a clear rubric. Theirs scored factual accuracy, citation accuracy, completeness, source quality and efficiency, each from 0 to 1.

Scout's exam has two files, and **you create both by hand**, copying them from Appendix D:

- `evals/questions.jsonl`: eight questions, one per line, each with a list of points a good answer must cover.
- `evals/run_eval.py`: runs Scout on each question, asks the lead model to grade the report against the findings with a five-part rubric, prints a score per question and an overall `SCORE`, and adds a line to `evals/results.tsv`.

Why by hand? Your Chapter 6 guard forbids Claude from editing these two files, and it is right to. In Chapter 14 an agent will be graded by them. The exam must be set by someone other than the student.

Also create an empty `evals/__init__.py` so Python treats the folder as a package.

Build it: an examiner subagent

A full evaluation prints a lot and takes several minutes. That is exactly the kind of work to keep off your main desk. Create `.claude/agents/eval-runner.md` from Appendix D. It runs `python -m evals.run_eval --limit 3`, compares the score with the previous one in `results.tsv`, and reports back in at most eight lines. It uses Haiku, since reading a table needs no great intelligence.

Then:

```
Use the eval-runner subagent to score Scout.
```

This costs real money, roughly the price of three Scout questions plus grading, so well under a pound. When it finishes you have a baseline score, probably somewhere between 0.6 and 0.9. Write it down. From now on, any change you make to Scout can be judged by whether this number goes up.

Checkpoint. Both subagents exist and work; the exam files are in `evals/`; `evals/results.tsv` has one line. Commit with "Reviewer, exam and examiner".

Check your understanding

1. What does a subagent hand back to the main conversation, and why is that useful?
2. How do you stop a subagent from being able to change files?
3. Why does the `description` field matter so much?
4. Why are the exam files written by hand rather than by Claude?
5. Anthropic started evaluating with about twenty questions. Why was such a small number enough at first?

Exercise. Write a third subagent, `prompt-critic`, with only the Read tool, that reads all four files in `scout/prompts/` and suggests the single change most likely to improve Scout's reports, with its reasoning. Run it, but do not apply the suggestion yet. Keep it for Chapter 14.

Sources: Create custom subagents; How we built our multi-agent research system.

Chapter 11. Teaching Claude your recipes

A skill is a written recipe that Claude Code keeps on the shelf and only takes down when it needs it. If you find yourself pasting the same instructions or checklist into Claude Code again and again, that text wants to be a skill.

How a skill works

A skill is a folder containing a file called `SKILL.md`. Like a subagent file, it starts with a few settings between `---` lines, followed by instructions in plain English:

```
---
name: add-tool
description: Adds a new tool that Scout's researchers can call.
Use when the user
asks to give Scout's researchers a new tool or ability.
---

# Adding a tool to Scout's researchers
1. Agree the tool's purpose in one sentence...
```

The clever part is what loads when. At the start of every session, Claude sees only each skill's **name and description**: a line or two. The full instructions load only when the skill is used. You can keep a dozen long, detailed recipes on hand and they cost almost nothing until one is needed. That is the big difference from CLAUDE.md, which is read in full every time.

A skill can be used in two ways:

- **You call it** by typing a slash and its name: `/add-tool`. Anything you type after the name is passed in, and the skill can place it in its instructions with `$ARGUMENTS`.
- **Claude calls it** on its own, when your request matches the description. Ask "give the researchers a way to look things up on Wikipedia" and Claude will reach for `add-tool` without being told.

Some recipes should never run on Claude's initiative: deploying, sending messages, anything with consequences. Add `disable-model-invocation: true` to the settings and only you can start it.

A skill folder can also hold supporting files, such as a long reference document or a script, which the instructions point to and Claude opens only if needed. And a line beginning with an exclamation mark and a command in backticks, such as !`git status --short` , is replaced by that command's output before Claude reads the skill, so the recipe arrives with fresh facts already in it.

Skills live in `.claude/skills/<name>/SKILL.md` for one project, or `~/.claude/skills/<name>/SKILL.md` for all of yours. They follow an open standard, Agent Skills, so the same file works in other tools that support it.

Which tool for which job?

You now have four ways to shape Claude Code. They are easy to confuse, so here they are side by side:

You want…	Use	Because
Facts Claude should always know about this project	CLAUDE.md	Loaded every session, in full
A procedure Claude should follow when a certain job comes up	A skill	Loaded only when needed; can be called with a slash
A noisy or specialised job done out of sight, with only a summary back	A subagent	Its own context, tools and instructions

You want...	Use	Because
Something that must always or never happen	A hook	Enforced by code, not by Claude's judgement

A reasonable rule of thumb: if a section of CLAUDE.md has grown into a list of steps, move it into a skill.

Build it: the add-tool recipe

Create `.claude/skills/add-tool/SKILL.md` from Appendix D. It is Chapter 7 boiled down into seven steps: agree the purpose, define the tool with a verb-first name and a description written for a new colleague, write a handler that returns short readable text and helpful errors, wire it into the researcher, mention it in the researcher's prompt, test it without touching the internet, and report back in plain English.

Now use it to give Scout a new ability. Wikipedia offers a free, public summary of any article, which makes a good second source of background facts.

```
/add-tool lookup_wikipedia: given an article title, return the
first paragraph of the
English Wikipedia summary and the article's web address, using
the public endpoint
https://en.wikipedia.org/api/rest_v1/page/summary/<title>.
```

Follow along as Claude works through the recipe's steps in order. When it is done, ask Scout a question in the browser where Wikipedia would obviously help, such as the history of a city, and check whether any findings come from wikipedia.org.

This tool is not in the reference code in Appendix D; it is yours. If you would rather not keep it, `git checkout .` throws the uncommitted changes away.

Build it: the ship recipe

Create `.claude/skills/ship/SKILL.md` from Appendix D. Look at three lines in its settings and body:

- `disable-model-invocation: true`: Claude may never decide by itself that it is time to ship.
- `allowed-tools: Bash(python -m pytest *) Bash(git status *) ...`: while the skill runs, these specific commands do not need your approval each time.
- `!`git status --short`` : the current list of changed files is pasted into the recipe before Claude reads it.

The recipe runs the tests, asks the code-reviewer subagent to look at the changes, checks that none of the cost limits in `config.py` has crept upwards, searches the changes for anything that looks like an API key, and commits. It deliberately stops short of pushing to GitHub; in Chapter 15 that push is what puts Scout live, and that decision stays with you.

Try it:

```
/ship
```

Checkpoint. Typing `/` in Claude Code lists `add-tool` and `ship`. Commit with "Skills: add-tool and ship" (or let `/ship` do it).

Check your understanding

1. At the start of a session, how much of each skill does Claude see? When does it see the rest?
2. What does `disable-model-invocation: true` do, and why does the ship skill need it?
3. What happens to a line like `!`git status --short`` in a skill?

4. Your CLAUDE.md has a twelve-step section on preparing a release. What should you do with it?
5. What is the difference between a skill and a subagent?

Exercise. Write a personal skill in `~/.claude/skills/explain/SKILL.md` that you can call as `/explain` followed by a file path. It should explain that file to a non-programmer in under 200 words, with one analogy, then list the three lines most worth understanding. Try it on `scout/graph.py`.

Source: Extend Claude with skills.

Chapter 12. Plugging into the world

The Model Context Protocol, MCP, is a shared plug shape for AI tools. Any program that speaks it, called an **MCP server**, can offer tools to any AI application that speaks it, called an **MCP client**. Claude Code is a client. So is the Claude desktop app, and so are many other AI products. Write a tool once as an MCP server, and every one of them can use it.

By the end of this chapter, Scout will be one. You will be able to sit in Claude Code, or in the Claude desktop app, and say "research this properly", and Claude will hand the question to your own research team.

The shape of MCP

An MCP server mostly offers **tools**, described exactly as in Chapter 7: a name, a description, an input schema. It can also offer **resources** (documents the client can read) and **prompts** (ready-made instructions), but tools are what you will use most.

Servers connect in one of two main ways:

Transport	Where the server runs	Example
stdio	On your own computer. The client starts it as a program and talks to it through its input and output	Scout's MCP server; a database tool
HTTP	On someone else's computer, reached over the internet, often with a sign-in	GitHub, Notion, Sentry and many other services

In Claude Code you add a server with `claude mcp add`, run in your ordinary terminal (not inside a Claude session):

```
# a remote server
claude mcp add --transport http notion https://mcp.notion.com/mcp

# a local program; everything after -- is the command that starts
it
claude mcp add myserver -- python /full/path/to/server.py
```

The `--scope` option chooses who gets it: `local` (the default: just you, just this project), `project` (written to a `.mcp.json` file in the project, which you can commit so everyone gets it), or `user` (just you, every project). Inside Claude Code, `/mcp` shows each server, whether it is connected, and lets you sign in to the ones that need it.

Once connected, each of the server's tools appears to Claude under a name like `mcp__notion__search`. The same name works in hook matchers and permission rules, so everything from Chapter 6 still applies.

Two cautions from the documentation. Only connect servers you trust, since a tool runs code or reaches into accounts on your behalf. And a server that fetches content from the web can bring back text written to trick an AI, a risk called **prompt injection**; the more tools an agent holds, the more that matters. Scout's own researchers read the web for this reason with only one action available to them: recording a finding.

Writing tools for strangers

When you write an MCP server, the agent calling your tool may be one you have never met, holding dozens of other tools. Every principle from *Writing effective tools for agents* becomes more important. The article was written with MCP servers in mind: it recommends naming tools so they stand apart, saying in the description when *not* to use the tool, and returning compact, meaningful results. Scout's single tool follows that advice:

```
mcp = MCPServer("scout")

@mcp.tool()
def deep_research(question: str) → str:
"""Research a question on the public web using Scout's team of
research agents.

Returns a Markdown report that answers the question, with
numbered citations
and a list of sources. Use this for questions that need several
up-to-date
sources, comparisons, or a written summary. Do not use it for
simple facts you
already know. It takes one to four minutes and costs a small
amount per call.

question: the full question, written as you would ask a human
researcher,
including any country, time period or other constraints.
"""
return run_research(question)["report"]
```

The text inside the triple quotes becomes the description Claude reads. It says what comes back, when to use it, when not to, and that it is slow and costs money, so that a well-behaved agent will not call it for trivia. The name `deep_research` is chosen to be distinct from any ordinary web search tool the client might also have.

The `mcp` Python package does the protocol work. One note if you read older tutorials: version 2 of the package renamed the class `FastMCP` to `MCPServer`.

Build it: Scout as a server

1. Ask Claude Code:

```
```

and paste the docstring from the listing above.

2. Register it with Claude Code. Leave Claude Code (`/exit`), make sure you are in the `scout` folder with `(.venv)` showing, and run:

```

```

On Windows PowerShell, use the full path you see from `pwd` instead of `$(pwd)`, and `.venv\Scripts\python.exe` for the Python part. Then check it:

```

```

`scout` should show as connected. The server finds your API key on its own, because `config.py` reads `.env` from the project folder.

3. Use it. Start Claude Code and type:

```

```

Claude asks permission to use the new tool. Allow it. A research run takes a minute or more; Claude Code moves long tool calls into the background so you can keep working, and picks up the result when it arrives.

4. Optionally, add Scout to the Claude desktop app. In its settings, open the *Developer* section and choose to edit the configuration. Add an entry like this, using your own full paths:

```

```

Restart the desktop app, and Scout is available in ordinary chats too.

Checkpoint. `claude mcp list` shows `scout` connected, and Claude Code saved a sourced report to `reports/`. Commit `scout/mcp_server.py` with "Scout as an MCP server".

Check your understanding

1. In MCP terms, is Claude Code a client or a server? What is Scout after this chapter?
2. When would you use the stdio transport, and when HTTP?
3. What does `--scope project` do, and why might a team want it?
4. Why does Scout's tool description say it is slow and costs money?

5. What is prompt injection, and why does it become a bigger risk as an agent gets more tools?

Exercise. Browse the Anthropic connector directory at claude.ai/directory and pick one service you already use. Read its tool list and descriptions. Find one description you think is well written and one you think is weak, and rewrite the weak one using Chapter 7's principles.

Sources: Connect Claude Code to tools via MCP; Writing effective tools for agents.

Part Five: Running on its own

Chapter 13. Work that outlasts a conversation

Every agent eventually runs out of context window. When it does, the next session starts with no memory of what the last one did. Anthropic's article *Effective harnesses for long-running agents* compares this to a project staffed by engineers working in shifts, where each new engineer arrives knowing nothing about the previous shift. The answer it gives is not a cleverer model; it is better handover notes. In this chapter you set up those notes for Scout and let a relay of Claude Code sessions add eight features while you do something else.

Two ways long jobs go wrong

The article describes two failures it saw when an agent was simply told to build a whole application.

- **Doing everything at once.** The agent tried to build the entire app in one go, ran out of context halfway through a feature, and left the next session a half-finished mess with no notes.
- **Declaring victory early.** A later session looked around, saw that plenty existed, and announced the job was done.

Both come from the same gap: nothing outside the agent's head said what "done" meant or where things stood.

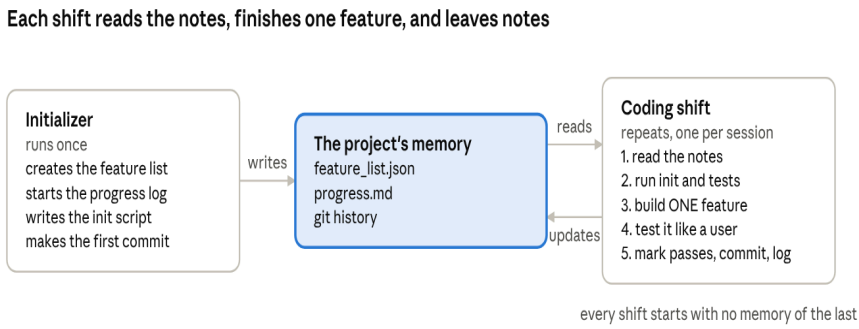
The fix: two roles and three files

The article's harness has an **initializer**, which runs once, and a **coding agent**, which runs again and again.

The initializer sets up three things and makes a first commit:

1. A **feature list**: every feature the finished app needs, each with the steps that prove it works, and every one marked as failing to begin with. This is the outside definition of done.
2. A **progress file**: a running log that each session appends to.
3. An **init script** that sets up the environment and checks the basics still work.

Each coding session then follows the same routine. It gets its bearings by reading the progress file, the recent Git history and the feature list. It runs the init script and fixes anything already broken before starting new work. It picks **one** unfinished feature, builds it, and tests it the way a user would; the article's agents used browser automation to click through the real app. Only then may it mark the feature as passing. It commits, writes a note for the next shift, and stops.



the long-running harness · one initializer, many shifts

No session remembers anything. Continuity lives entirely in three ordinary files that every shift reads first and updates last.

Two details from the article are worth copying exactly. The feature list is **JSON**, not a loose document, because the model was less inclined to rewrite or tidy a structured file. And the coding agent is told it may

change only the `passes` field: never delete a feature, never reword a test. Without that rule, the easiest way to finish is to quietly lower the bar.

A follow-up article in March 2026 went further, adding a separate evaluator agent to judge the work. Its reasoning: agents asked to grade their own output tend to praise it. Scout already applies that idea twice, in the reviewer and in the exam you cannot edit.

Running Claude Code without a person

So far you have always typed into Claude Code. It can also run a single instruction and exit, which is what a relay of shifts needs:

```
claude -p "your instructions here" --permission-mode acceptEdits
```

The `-p` flag means "print mode": do this task, print the result, finish. With nobody there to answer permission prompts, you decide in advance what is allowed. `acceptEdits` lets it change files in the project; `--allowedTools` lists the commands it may run without asking. Anything else is refused. Your hooks still run, so the guard and the test check from Chapter 6 still apply.

Build it: the harness

1. Copy the `harness` folder from Appendix D into your project: `feature_list.json`, `progress.md`, `init.sh`, `coding_prompt.md` and `run.sh`. Read `feature_list.json`. These are eight small, useful features, from "download report as Markdown" to "mark interrupted jobs as failed when the server restarts", each with its proof steps and `"passes": false`. You have played the initializer's part by setting them up; in a bigger project you would ask an initializer session to draft the list from a description of the app and then review it yourself.

2. Read `coding_prompt.md`. It is the article's routine in nine plain steps, ending with "Stop. Do not start a second feature."
3. Read `run.sh`. It loops for as many shifts as you ask for, stops early when nothing is left to do, and starts each shift as a fresh `claude -p` session with the coding prompt.
4. Make the scripts runnable and commit the harness so the first shift starts from a clean history:

```
```

On Windows, run the harness from Git Bash (installed with Git for Windows) or from WSL.

Build it: run one shift and inspect it

```
bash harness/run.sh 1
```

This takes several minutes and prints Claude's final summary. Then check its work as the next shift would:

```
git log --oneline -5
tail -20 harness/progress.md
grep -c '"passes": true' harness/feature_list.json
```

You should see a commit for feature 1, a new progress entry, and a count of 1. Start the server and try the new Download button yourself. If the shift marked the feature as passing but it does not work, you have found the "declaring victory early" failure in the wild; fix it with Claude interactively, and add a sentence to `coding_prompt.md` to prevent it.

Build it: let it run

When one shift looks trustworthy, run three, and go and make a cup of tea:

```
bash harness/run.sh 3
```

Come back and read `progress.md` from the top. It reads like a ship's log: each shift says what it found, what it did and what it left for the next. That log, the feature list and the Git history are the whole memory of the project. No single session needed to remember anything.

Checkpoint. At least four features pass, each with its own commit and progress entry, and the full test suite passes. Try each finished feature in the browser.

Check your understanding

1. What is the "shift worker" problem, in your own words?
2. Name the three files the initializer creates and what each is for.
3. Why may the coding agent change only the `passes` field?
4. Why does each shift work on exactly one feature?
5. What does `claude -p` do, and how do you control what it is allowed to do with no one watching?

Exercise. Add a ninth feature of your own to `feature_list.json`, with clear proof steps and `"passes": false`. Run one shift. Did it pick up your feature at the right time, and did your proof steps turn out to be specific enough?

Sources: Effective harnesses for long-running agents; Run Claude Code programmatically.

Chapter 14. The agent in a loop

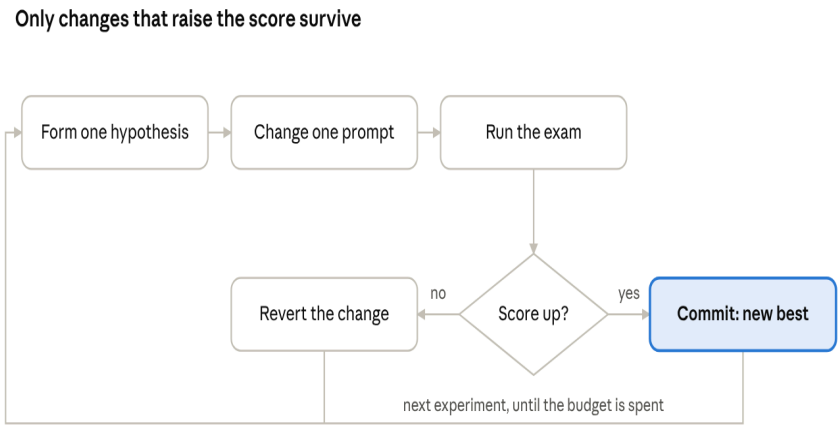
In March 2026 Andrej Karpathy published a small repository called *autoresearch*. It sets a coding agent loose on a machine-learning experiment overnight: change the code, run a five-minute test, keep the change if the score improved, throw it away if not, repeat. By morning there is a history of tested improvements and a log of everything that was tried. In this chapter you point the same loop at Scout's prompts.

How autoresearch works

The repository is deliberately tiny. Its three important files divide the work between human, agent and referee:

File	Who owns it	What it is
`program.md`	The human	Instructions for the agent, in plain English: what it may change, how to measure, what to do when things crash. Karpathy describes it as a very lightweight skill
`train.py`	The agent	The one file the agent is allowed to edit
`prepare.py`	Nobody	Data and the scoring function. Fixed, so the agent cannot change how it is judged

Every experiment runs for the same fixed time budget, so results are comparable. The score is a single number. The loop works like a ratchet: an improvement is committed with Git and becomes the new starting point; anything else is reverted, so the code can only get better by the measure being used. The instructions also carry a sense of taste: a small gain that adds ugly complexity is not worth keeping, while simplifying something without losing performance always is.



the autoresearch ratchet · keep or revert, then go again

Git is the ratchet: kept changes become commits and the new baseline, rejected ones are thrown away, so the score can only move one way.

The whole method is written in Markdown. There is no framework. A capable agent, clear written rules, a fixed referee and Git turn out to be enough.

Why this fits Scout

Anthropic reached the same idea from a different direction. Their research team found that Claude could diagnose why an agent was failing and suggest prompt fixes, and they built an agent that rewrote a tool's description after testing it dozens of times, which cut later agents' task times by 40 percent. *Writing effective tools for agents* describes the same move: hand Claude the evaluation transcripts and let it improve the tools.

Scout is ready for this, because you built every piece in earlier chapters:

autoresearch	Scout	Built in
`train.py`, the file the agent edits	`scout/prompts/*.md`	Chapters 3, 7 and 8
`prepare.py`, the fixed referee	`evals/questions.jsonl` and `evals/run_eval.py`	Chapter 10
The rule that the referee cannot change	The guard hook	Chapter 6
The score, lower is better	`SCORE`, higher is better	Chapter 10
`program.md`	`program.md`	This chapter

Two important differences

It costs money. Karpathy's loop burns electricity on a graphics card you already own. Every experiment of yours runs Scout on several questions and pays for each. So Scout's `program.md` sets a number of experiments and tells the agent to stop and report when it reaches it, where Karpathy's tells the agent never to stop. Each experiment with four questions costs very roughly the price of five or six Scout questions.

The score is noisy. Karpathy's training measurement is quite steady. Scout's is not: the web changes, the researchers search differently each time, and the grader is itself a model. Run the baseline twice and you will see the score move a little on its own. So `program.md` treats a change of less than 0.02 as no change. A loop that keeps every lucky wobble would fill your history with noise.

There is a third, quieter risk. If the loop tunes the prompts until they ace eight particular questions, it may have learned those questions rather than how to research. Anthropic guarded against this with held-out tests, questions kept back and never used for tuning. You will do the same at the end.

Build it

1. Copy `program.md` from Appendix D into the project folder and read it. Note what it permits (only the prompt files), the setup (a new branch and a baseline), the loop (one hypothesis, one small change, test, keep or revert, log it), the taste rules and the stopping point.
2. Make sure everything is committed: `git status` should be clean. The loop's reverts discard uncommitted changes to the prompts.
3. Measure the noise first. Run the baseline twice:

```
```

If the two scores differ by more than 0.02, raise the threshold in `program.md` to match. You have just calibrated your instrument.

4. Start the loop. You can watch it interactively, which is a good idea the first time:

```
```

Or run it unattended, as in Chapter 13:

```
```

If you remember your `prompt-critic` subagent's suggestion from Chapter 10, you can add it: "Try this hypothesis first: ...".

5. Read what happened:

```
```

(If `git branch` shows your main branch as `master`, use that name instead.) Each kept experiment is a commit with its hypothesis and score in the message. The diff shows exactly how the prompts changed.

6. Test on fresh questions. Write three new questions of your own, of the kind you actually want Scout for, and ask them in the browser on this branch. Are the answers better than before? If the score

rose but your fresh answers did not improve, the loop learned the exam. Throw the branch away.

7. If you are happy, bring the improvements home:

```
```

Checkpoint. `evals/experiments.md` lists your experiments with scores; any kept change is a commit on the autoresearch branch; you have judged it on questions the loop never saw.

Check your understanding

1. In autoresearch, why is `prepare.py` off limits to the agent? What plays that role in Scout, and what enforces it?
2. Explain the ratchet in two sentences.
3. Why does Scout's `program.md` stop after a set number of experiments when Karpathy's says never stop?
4. Why is a change of less than 0.02 treated as no change?
5. What is a held-out question, and what problem does it catch?

Exercise. Change `program.md` so the agent may also edit the effort limits in `scout/config.py`, but only downwards. Add a rule that a change which keeps the score the same while lowering cost counts as an improvement. Run two experiments. Did it find a cheaper Scout that is just as good?

Sources: karpathy/autoresearch; How we built our multi-agent research system; Writing effective tools for agents.

Chapter 15. Shipping Scout

An app on your laptop is a prototype. An app on the internet that strangers cannot run up a bill on is a product. This chapter takes Scout the last mile: a password, a daily limit, a home on GitHub, and a server on Railway, a hosting service that runs your code from GitHub and gives it a web address.

Anthropic's research article ends with a line every builder learns eventually: in agent systems, the last mile often turns out to be most of the journey. The good news is that most of Scout's last mile is already behind you. The tests, limits, hooks and evaluation you built along the way are exactly what makes it safe to publish.

Build it: lock the doors

Once Scout has a public address, anyone who finds it could use it, with your API key and your money. Two protections go in before it leaves your computer.

```
Add two settings to scout/config.py, read from the environment:
SCOUT_PASSWORD
(default empty) and SCOUT_DAILY_LIMIT (default 25). In
scout/server.py:
1. If SCOUT_PASSWORD is set, every route except /health asks the
browser for a
username and password with HTTP Basic auth. The username is
"scout". Compare with
secrets.compare_digest.
2. POST /api/research returns 429 with a friendly message once
count_jobs_today()
reaches the daily limit.
Add tests for both. Also create .env.example listing every
setting with a comment,
and a Procfile: web: uvicorn scout.server:app --host 0.0.0.0 --
port ${PORT:-8000}
```

Then set a password locally to try it: add `SCOUT_PASSWORD=something-long` to your `.env` by hand, restart the server, and reload the page. Your browser should ask for a username (`scout`) and the password.

Finally, go back to platform.claude.com and check the monthly spending limit you set in Chapter 2. That limit is your last line of defence, and it works even if everything else fails.

Build it: a final review

Run your own release checklist:

```
/ship
```

The skill runs the tests, has the reviewer look over the changes, checks that no cost limit has crept upwards and that no key has slipped into the code, then commits. Fix anything it raises.

Build it: put the code on GitHub

GitHub keeps a copy of your project online, and Railway will read it from there.

1. Install the GitHub command-line tool from cli.github.com, then sign in:

```

```

Choose GitHub.com, HTTPS, and log in with a web browser.

2. Create a **private** repository from your project and upload it in one step:

```

```

3. Open the repository on github.com and check two things: the code is there, and **there is no `.env` file**. If you ever do find your key on GitHub, delete the key immediately on platform.claude.com and

create a new one. Deleting the file afterwards is not enough, because Git remembers history.

Build it: deploy to Railway

1. Go to railway.com and sign in with your GitHub account.
2. Create a new project and choose to deploy from a GitHub repository. Pick `scout`. Railway notices `requirements.txt`, installs the packages, and uses the `Procfile` to start the server.
3. Open the service's **Variables** and add:
   - `ANTHROPIC_API_KEY`: your key
   - `SCOUT_PASSWORD`: a long password
   - `SCOUT_DAILY_LIMIT`: for example `25`
   - `SCOUT_DB`: `/data/scout.db`
4. Add a **volume** to the service and mount it at `/data`. A hosted server's own disk is wiped every time it redeploys; the volume is a separate disk that survives, so your past reports are kept. That is what `SCOUT_DB` points at.
5. In the service's networking settings, generate a public domain. Railway gives you an address ending in `.up.railway.app`.
6. Optionally, set the health check path to `/health`, so Railway knows when the new version is ready before switching traffic to it.

Open the address. Your browser asks for the password; enter `scout` and yours. Ask a question. When the report arrives, Scout is live.

Railway charges by usage; check its pricing page for current plans. If a deploy fails, the deployment's **logs** say why, and pasting them into Claude Code is usually the fastest way to a fix.

Living with it

From now on, changing Scout follows one path, every time:

1. Make the change with Claude Code, in plan mode first if it is more than a line or two.
2. Run the evaluation through the eval-runner subagent if the change touches prompts, tools or the workflow. Keep the change only if the score holds or rises.
3. Run `/ship`.
4. Type `git push` yourself. Railway sees the new commit and redeploys automatically within a couple of minutes.

Keep an eye on two numbers: your usage on platform.claude.com, and the questions you ask Scout that it answers badly. Add the worst of those to a new held-out list, and every so often promote one into the exam.

Where to go next

You now know enough to extend Scout in directions this book did not take. A few that use exactly what you have learned:

- **Routing**, the one pattern from Chapter 1 that Scout does not use: a quick first call that sends simple factual questions straight to a single researcher, skipping the full team.
- **More sources** through `/add-tool` or MCP: your company's documents, a news archive, an academic search service.
- **A checkpointer** in LangGraph, so a long run survives a server restart and can pause for your approval before the report is written.
- **A second harness run** with features of your own choosing.

The principles underneath will not change as fast as the tools. Start simple. Make the agent's work visible. Spend real care on its tools. Measure before you believe. Keep the referee out of the player's reach. And write things down, because neither you nor your agents will remember them otherwise.

Final test

1. Name the two protections that stop strangers from spending your money, and the third that works even if both fail.
2. Why must the database live on a Railway volume?
3. Your API key appears in a GitHub commit. What do you do first, and why is deleting the file not enough?
4. Describe the four-step path for every future change to Scout.
5. Pick any three chapters and, for each, name the main idea and the file in Scout where it lives.

Final exercise. Ask Scout, live on the internet, a question that genuinely matters to you this week. Read the report as a sceptic: check three citations, look for anything missing. Then decide one improvement, make it using the path above, and ship it. That is the whole job, and now you can do it.

Sources: How we built our multi-agent research system; Railway documentation; GitHub CLI.

Appendices

Appendix A. Answers to the tests

Short answers only. If yours says the same thing in different words, it is right.

Chapter 1. (1) In a workflow, your code decides the steps; in an agent, the model decides them as it goes. (2) Ordinary software, the harness, runs the tool and passes the result back; the model only asks. (3) Retrieval, tools and memory. (4) Evaluator-optimizer. (5) Each agent has its own context window, so the team reads and reasons about far more in parallel; but it uses many more tokens, roughly fifteen times a chat, so it only pays for valuable, broad questions.

Chapter 2. (1) Scout is a separate program calling the Claude API directly, which is billed to an API account, not your subscription. (2) The project's virtual environment is active; if it is missing, `cd` into the folder and run the activate command. (3) Once stops Claude mid-action; twice opens the rewind menu to go back to an earlier point. (4) Normal, accept edits, and plan; plan mode is for looking and planning before any change. (5) So the key never passes through Claude, and because later you forbid Claude from touching that file.

Chapter 3. (1) Show the prompt to someone who knows nothing about the task: could they do it well from that alone? (2) Without effort rules the planner overspends on simple questions, which Anthropic saw in its own system. (3) To separate different kinds of material so Claude can tell them apart, e.g. `<question> ... </question>`. (4) Planning and writing need judgement (Sonnet); searching is many simple calls where speed and price matter (Haiku). (5) For example: "I'm deciding whether my next car should be electric. In under 300 words, for a UK driver doing 8,000 miles a year, compare running costs, charging at home and range with a petrol car, and end with the main question I should ask a dealer."

Chapter 4. (1) Explore, plan, build, check; plan mode supports planning. (2) Research takes minutes; the page should stay responsive, so it starts a background job and polls it. (3) Type `@scout/server.py` in your message. (4) Rewind undoes Claude's recent file edits within the session; a Git commit is a permanent save point you can return to any time. (5) An exact description of what you saw, and the red error lines from the browser's console.

Chapter 5. (1) You write CLAUDE.md as instructions; Claude writes auto memory as notes from your corrections. (2) `~/.claude/CLAUDE.md`. (3) Long files use context and make each instruction less likely to be followed. (4) The rule loads only when Claude works on matching files. (5) No: CLAUDE.md is advice Claude tries to follow, not enforced. A hook enforces it.

Chapter 6. (1) Run every time, and block an action or stop Claude finishing. (2) No; exit 1 counts as a broken hook and the action goes ahead. Use exit 2, or a JSON deny. (3) To avoid sending Claude back forever; after one push back, it lets Claude stop. (4) `.claude/settings.json` in the project, committed to Git. (5) Later an agent is graded by them; the student must not be able to rewrite the exam.

Chapter 7. (1) The name, the description, and the input schema. (2) Listing returns everything and wastes context; searching returns only what is relevant. (3) A server tool runs on Anthropic's servers (`web_search`); a client tool runs in your code (`record_finding`). (4) It offers only the `submit_plan` tool, asks for it in the prompt, and reminds Claude once if it answers in prose. (5) So a confused agent can never loop forever and run up costs.

Chapter 8. (1) Searching means reading a lot and keeping a little; separate context windows let each researcher read widely and hand back only the distilled findings. (2) The model cannot then invent or mistype a web address. (3) A mechanical citation check in code, then a model reading the draft against the findings. (4) Each researcher

mostly waits for Claude's reply, so the waits overlap. (5) For example: vague delegation, guarded by objectives and boundaries; content-farm sources, guarded by the researcher prompt's source advice.

Chapter 9. (1) State is the shared notebook (question, findings, report...); nodes are steps (plan, research, write, review); edges are arrows, such as research to write. (2) It starts one node per item and runs them in parallel; Scout uses it to start a researcher per task. (3) Parallel researchers would overwrite each other's findings. (4) Unchanged tests passing is evidence the rewrite kept the behaviour. (5) It saves state after each step so a run can pause or resume after failure; agents are long-running and errors compound, so restarting from scratch is costly.

Chapter 10. (1) A summary, keeping the clutter of its work out of your context. (2) Leave Edit and Write out of its `tools`. (3) Claude uses it to decide when to delegate. (4) The agent will be graded by them; it must not set its own exam. (5) Early changes have large effects, so a few examples show them clearly.

Chapter 11. (1) Only the name and description; the full instructions load when the skill is used. (2) Only you can start it; shipping has consequences and should never be Claude's decision. (3) The command runs and its output replaces the line before Claude reads the skill. (4) Move it into a skill. (5) A skill is instructions used in your main conversation; a subagent is a separate worker with its own context that reports back.

Chapter 12. (1) Claude Code is a client; Scout is now a server. (2) stdio for a program running on your own computer; HTTP for a service elsewhere on the internet. (3) It writes the server to `.mcp.json` in the project, which can be committed so the whole team gets it. (4) So agents do not call a slow, paid tool for trivial questions. (5) Text in fetched content written to trick an AI; each extra tool is another thing a tricked agent could misuse.

Chapter 13. (1) Each session starts with no memory of the last, like a shift worker arriving with no handover. (2) Feature list (definition of done), progress file (log for the next shift), init script (set up and check basics). (3) Otherwise the easiest way to finish is to lower the bar by editing the tests. (4) Working on everything at once runs out of context mid-feature and leaves a mess. (5) It runs one task without a person and exits; you set `--permission-mode` and `--allowedTools` beforehand, and hooks still apply.

Chapter 14. (1) It holds the scoring; if the agent could change it, it could raise its score without improving. In Scout the evaluation files play that role, enforced by the guard hook. (2) Every change is tested; only improvements are committed and become the new baseline, so the measure can only move one way. (3) Every experiment costs real money. (4) The score varies by chance; smaller changes are probably noise. (5) A question never used for tuning; it catches a loop that learned the exam rather than the skill.

Chapter 15. (1) The password and the daily limit; the Console spending limit works even if both fail. (2) A hosted server's own disk is wiped on every redeploy. (3) Delete the key on platform.claude.com and make a new one; Git history keeps old versions of files. (4) Change with Claude Code; evaluate if it touches behaviour; `/ship`; `git push`. (5) For example: Chapter 6, checks that can say no, in `.claude/hooks/protect.py`; Chapter 7, tool design, in `scout/tools.py`; Chapter 9, the workflow graph, in `scout/graph.py`.

Appendix B. Glossary

Term	Meaning
Agent	A model that uses tools in a loop, deciding its own next step, until a task is done
API key	A secret that lets a program use your Anthropic account and be billed for it
Context window	Everything a model can see at once; its working memory
Evaluation (eval)	A fixed set of tasks and a way of scoring the results, used to tell whether a change helped
Harness	The ordinary code around a model that runs its tools, manages its loop and keeps its records
Hook	A program Claude Code runs automatically at a set moment, which can block actions
JSON	A simple text format for structured data: names and values in braces
LangGraph	A library for building agent systems as graphs of steps sharing a state
MCP	Model Context Protocol, a standard way for AI applications to use tools from other programs
Prompt	The text you give a model; the system prompt is the standing briefing above the conversation
Skill	A recipe file Claude Code loads only when needed, or when you call it with a slash
Subagent	A separate Claude with its own context and tools that does a task and reports back
Test	A small program that checks another program behaves as it should
Token	The unit models read and write in, roughly three-quarters of a word

Term	Meaning
Tool	An action a model can ask for, described by a name, a description and an input schema
Virtual environment	A private folder of Python packages for one project
Workflow	A system where code fixes the steps in advance and models fill them in

Appendix C. When things go wrong

Symptom	Likely cause	Fix
`claude: command not found`	The terminal has not picked up the new install	Open a new terminal window; run `claude doctor`
`No module named ...`	The virtual environment is not active	`cd scout`, then the activate command from Chapter 2
Authentication error from the API	`.env` missing, misnamed or the key mistyped	Check the file is called exactly `.env` in the project folder and starts `ANTHROPIC_API_KEY=`
Error mentioning web search not enabled or not allowed	Web search is off for your organisation	Turn it on in the Console settings
Error that a model is not found	Model names change over time	Set `SCOUT_LEAD_MODEL` and `SCOUT_WORKER_MODEL` in `.env` to current names from the models overview page
Hooks do nothing	Wrong Python name, or settings not reloaded	On Windows use `python` in `.claude/settings.json`; restart Claude Code; check `/hooks`
Claude keeps being sent back by the Stop hook	A test is failing	Read the failure it shows; ask Claude to explain it in plain English before fixing
`ImportError` mentioning `FastMCP`	Old tutorial code for version 1 of the `mcp` package	Use `from mcp.server.mcpserver import MCPServer`
Harness script fails on Windows	Bash scripts need a Bash shell	Run from Git Bash or WSL; in Git Bash the activate path is `.venv/Scripts/activate`
Railway deploy fails	Usually a missing variable or package	Read the deploy logs and paste them to Claude Code

Symptom	Likely cause	Fix
Reports vanish after a redeploy	The database is not on a volume	Mount a volume at `/data` and set `SCOUT_DB=/data/scout.db`
A question costs more than expected	Broad question, many searches	Lower the limits in `scout/config.py`; check the planner's effort rules

Appendix D. Scout's complete source

This is the finished reference version of Scout, every file, tested. The same files come as a download with this book (`scout-reference.zip`). To bring one of your files into line with the reference, copy the listing and paste it into Claude Code after the words *"Replace this file with exactly this content:"*, or paste it into the file yourself in your editor. The two evaluation files and `.env` are the exceptions: create those by hand, as Chapters 2 and 10 explain.

Project layout

```
scout/
├── .claude/
│   ├── agents/code-reviewer.md Chapter 10
│   ├── agents/eval-runner.md Chapter 10
│   ├── hooks/protect.py Chapter 6
│   ├── hooks/run_tests.py Chapter 6
│   ├── settings.json Chapter 6
│   └── skills/add-tool/SKILL.md, skills/ship/SKILL.md Chapter 11
├── evals/ __init__.py, questions.jsonl, run_eval.py Chapter 10
├── harness/ feature_list.json, progress.md, init.sh,
 coding_prompt.md, run.sh Chapter 13
├── scout/
│   ├── __init__.py, config.py, llm.py, prompts.py Chapter 3
│   ├── prompts/planner.md Chapter 3
│   ├── prompts/researcher.md, tools.py Chapter 7
│   ├── prompts/writer.md, prompts/reviewer.md Chapter 8
│   ├── agents.py Chapters 7 and 8
│   ├── graph.py Chapters 4, 8 and 9
│   ├── store.py, server.py, static/index.html Chapters 4 and 15
│   └── mcp_server.py Chapter 12
├── tests/ __init__.py, conftest.py, fake_llm.py, test_tools.py,
 test_graph.py, test_server.py, test_eval.py
├── CLAUDE.md Chapter 5
├── program.md Chapter 14
├── requirements.txt, .env.example, .gitignore, Procfile,
.python-version
└── .env your key; never committed, never shown to Claude
```

The files `scout/__init__.py`, `tests/__init__.py` and `evals/__init__.py` are empty.

requirements.txt

```
anthropic≥2.0
langgraph≥1.0
fastapi≥0.115
uvicorn≥0.30
python-dotenv≥1.0
mcp≥2.0
pytest≥8.0
httpx≥0.27
```

.env.example

```
# Copy this file to .env and fill in your own values.
# Never commit .env to GitHub.

# Your Anthropic API key, from platform.claude.com > API keys
ANTHROPIC_API_KEY=sk-ant-your-key-here

# Models. Leave these as they are unless the book tells you
otherwise.
SCOUT_LEAD_MODEL=claude-sonnet-5-5
SCOUT_WORKER_MODEL=claude-haiku-5-5

# Optional: set a password to protect Scout when it is on the
internet.
# The username is always "scout".
SCOUT_PASSWORD=

# Optional: the most questions Scout will accept in one day
(protects your bill).
SCOUT_DAILY_LIMIT=25
```

.gitignore, Procfile and .python-version

```
.env
.venv/
__pycache__/
*.pyc
scout.db
.pytest_cache/
.claude/settings.local.json
CLAUDE.local.md
web: uvicorn scout.server:app --host 0.0.0.0 --port ${PORT:-8000}
```

`.python-version` contains one line: `3.12`.

CLAUDE.md

```
# Scout

Scout is a research assistant web app. A user asks a question; a
planner agent splits it into tasks,
parallel researcher agents search the web and record findings, a
writer turns findings into a cited
Markdown report, and a reviewer checks it. Built with Python,
FastAPI, LangGraph and the Claude API.

## Commands
- Set up: `python3 -m venv .venv && source .venv/bin/activate &&
pip install -r requirements.txt`
- Run the app: `uvicorn scout.server:app --reload` then open
http://127.0.0.1:8000
- Run tests (free, no API calls): `python -m pytest -q`
- Run the evaluation (costs money, calls Claude): `python -m
evals.run_eval --limit 3`

## Layout
- `scout/graph.py` the LangGraph workflow: plan → research
(parallel) → write → review
- `scout/agents.py` the four agents; `scout/tools.py` tool
definitions and handlers
- `scout/prompts/*.md` every prompt Scout uses, as plain text
- `scout/server.py` web server; `scout/static/index.html` the
single web page
- `scout/store.py` SQLite storage; `scout/mcp_server.py` Scout as
an MCP tool
- `tests/` uses a fake Claude (`tests/fake_llm.py`), so tests
never need an API key
- `evals/` the exam: questions plus an AI grader

## Rules
- Never read, print or edit `.env`. It holds the API key.
- Never edit `evals/questions.jsonl` or `evals/run_eval.py`; they
are the referee.
- Every new feature needs a test in `tests/` that uses the fake
client, not the real API.
- Keep prompts in `scout/prompts/`, not inside Python code.
- Explain changes in plain English: the owner of this project is
not a programmer.
```

scout/config.py

```
"""Settings for Scout. Values come from the .env file or the
environment."""
import os
from pathlib import Path

from dotenv import load_dotenv

PROJECT_ROOT = Path(__file__).resolve().parent.parent
load_dotenv(PROJECT_ROOT / ".env")

# The lead model plans, writes and reviews. The worker model does
the searching.
LEAD_MODEL = os.getenv("SCOUT_LEAD_MODEL", "claude-sonnet-5-5")
WORKER_MODEL = os.getenv("SCOUT_WORKER_MODEL", "claude-haiku-5-
5")

# Effort limits. These keep a single question from running up a
large bill.
MAX_SUBTASKS = 5 # most researchers the planner may start
MAX_RESEARCH_TURNS = 8 # most back-and-forth turns for one
researcher
MAX_SEARCHES_PER_RESEARCHER = 5
MAX_FINDINGS_PER_RESEARCHER = 8
MAX_REVISIONS = 1 # how many times the writer may redo a rejected
report

DB_PATH = os.getenv("SCOUT_DB", str(PROJECT_ROOT / "scout.db"))
PASSWORD = os.getenv("SCOUT_PASSWORD", "")
DAILY_LIMIT = int(os.getenv("SCOUT_DAILY_LIMIT", "25"))
```

scout/llm.py

```
"""One shared connection to the Claude API.

Tests swap in a fake client with set_client(), so they run
without
an API key and without spending money.
"""
import anthropic

_client = None


def client():
global _client
if _client is None:
_client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY
return _client


def set_client(new_client):
global _client
_client = new_client
```

scout/prompts.py

```
"""Loads Scout's prompts from the scout/prompts folder.

The prompts live in plain text files so you can read and edit
them
without touching any code (and so an agent can tune them in
Chapter 14).
"""
from pathlib import Path

PROMPTS_DIR = Path(__file__).resolve().parent / "prompts"


def load_prompt(name: str) → str:
return (PROMPTS_DIR / f"{name}.md").read_text(encoding="utf-8")
```

scout/prompts/planner.md

As printed in Chapter 3.

scout/prompts/researcher.md

```
You are a researcher working for Scout. You have been given one
research task, which is part of a larger question. Other
researchers are handling the other parts, so stay inside your
task's boundaries.

You have two tools:
- web_search: searches the web. Start with short, broad queries
(two to five words), look at what comes back, then make narrower
queries. Do not add a year to a query unless the task is about a
specific year.
- record_finding: saves one fact you have found, with the page it
came from. Only recorded findings reach the final report.
Anything you do not record is lost.

How to work:
1. Search. Read the results critically. Prefer primary sources:
official sites, government and company publications, research
papers, and established news organisations. Treat content farms,
anonymous blogs and marketing pages with suspicion.
2. Each time you find a fact that answers part of your task, call
record_finding straight away. One fact per call. Write the claim
in your own words as a complete sentence that makes sense on its
own.
3. If two good sources disagree, record both and say in each
claim who says what.
4. Stop when your task is answered, when you have recorded about
five strong findings, or when further searching is not turning up
anything new. Do not keep searching for the sake of it.

When you have finished, reply with one or two sentences
summarising what you found and anything you could not find.
```

scout/prompts/writer.md

```
You are the writer for Scout, a research assistant. Researchers
have gathered findings from the web. Your job is to turn them
into a clear report that answers the user's question.

Rules:
- Use only the findings provided. Do not add facts from your own
memory. If the findings do not answer part of the question, say
plainly that Scout could not find it.
- Every factual sentence must end with a citation in square
brackets giving the source number, like this: [2]. If a sentence
draws on two sources, write [2][5]. Use only source numbers that
appear in the findings.
- Where sources disagree, say so and cite both sides.
- Lead with the answer. The first paragraph should answer the
question directly in two to four sentences.
- After that, add short sections with headings (using ##) only if
the question has several parts.
- Write in plain English for an intelligent reader who is not an
expert. Keep it under 600 words.
- Do not write a "Sources" section; Scout adds that
automatically.

Write the report in Markdown, inside <report> tags.
```

scout/prompts/reviewer.md

```
You are the fact-checker for Scout, a research assistant. You
will be given the user's question, the list of findings the
researchers gathered (each with a source number), and a draft
report.

Check the draft against the findings, not against your own
knowledge:
1. Does the first paragraph answer the question directly?
2. Is every factual sentence backed by a citation such as [3]?
3. Does each citation point to a source whose finding actually
supports that sentence?
4. Does the report claim anything that none of the findings
support?
5. Does it cover every part of the question that the findings
make it possible to answer?

Approve the report if it has no serious problems. Small matters
of style are not a reason to reject it. If you reject it, list
each problem as a short, specific instruction the writer can act
on, for example: "Paragraph 2 says the bridge opened in 1998 but
cites [4], which says 1999."

Call the submit_review tool with your verdict.
```

scout/tools.py

```
"""The tools Scout's agents can use.

A tool is a description Claude reads (name, purpose, inputs)
plus, for
our own tools, a small Python function that does the work when
Claude
calls it. The descriptions matter as much as the code: Claude
decides
when and how to use a tool from its description alone.
"""
from urllib.parse import urlparse

from . import config

# Web search runs on Anthropic's servers. We only describe it;
Claude
# performs the searches itself and the results come back in its
reply.
WEB_SEARCH_TOOL = {
"type": "web_search_20250305",
"name": "web_search",
"max_uses": config.MAX_SEARCHES_PER_RESEARCHER,
}

RECORD_FINDING_TOOL = {
"name": "record_finding",
"description": (
"Save one fact you found during research so it can be used in the
final report. "
"Call this once per fact, as soon as you find it. Only recorded
findings reach the "
"report; anything not recorded is lost. Do not record opinions or
guesses, and do "
"not record the same fact twice."
),
"input_schema": {
"type": "object",
"properties": {
"claim": {
"type": "string",
"description": (
"The fact, written as one complete sentence in your own words
that makes "
"sense without any other context. Include names, numbers and
```

```
dates. "
"Example: 'The Golden Gate Bridge opened to traffic on 28 May
1937.'"
),
},
"source_url": {
"type": "string",
"description": "The full web address (starting with https://) of
the page that states this fact.",
},
"source_title": {
"type": "string",
"description": "The title of that page, or the name of the
organisation that published it.",
},
},
"required": ["claim", "source_url", "source_title"],
},
}

SUBMIT_PLAN_TOOL = {
"name": "submit_plan",
"description": "Submit the research plan: the list of independent
research tasks for the researchers.",
"input_schema": {
"type": "object",
"properties": {
"subtasks": {
"type": "array",
"minItems": 1,
"maxItems": config.MAX_SUBTASKS,
"items": {
"type": "object",
"properties": {
"title": {"type": "string", "description": "A short name for the
task, three to six words."},
"objective": {"type": "string", "description": "Exactly what to
find out, and what is out of scope."},
"search_hints": {
"type": "array",
"items": {"type": "string"},
"description": "Two or three short starting search phrases.",
},
},
"required": ["title", "objective", "search_hints"],
},
```

```
}
},
"required": ["subtasks"],
},
}

SUBMIT_REVIEW_TOOL = {
"name": "submit_review",
"description": "Submit your verdict on the draft report.",
"input_schema": {
"type": "object",
"properties": {
"approved": {"type": "boolean", "description": "true if the
report can be published as it is."},
"problems": {
"type": "array",
"items": {"type": "string"},
"description": "Specific problems for the writer to fix. Empty if
approved.",
},
},
"required": ["approved", "problems"],
},
}


def handle_record_finding(tool_input: dict, findings: list,
subtask_title: str) → str:
"""Runs when a researcher calls record_finding.

Returns a short message that Claude reads. Error messages say
exactly
what was wrong and how to fix it, so the agent can correct
itself.
"""
claim = (tool_input.get("claim") or "").strip()
url = (tool_input.get("source_url") or "").strip()
title = (tool_input.get("source_title") or "").strip() or url

if len(claim) < 15:
return "Not saved: the claim is missing or too short. Write the
fact as one complete sentence."
parsed = urlparse(url)
if parsed.scheme not in ("http", "https") or not parsed.netloc:
return (
"Not saved: source_url must be a full web address starting with
```

```
https://, "
f"for example https://www.example.org/page. You sent: {url!r}"
)
if len(findings) ⩾ config.MAX_FINDINGS_PER_RESEARCHER:
return (
f"Not saved: you have already recorded {len(findings)} findings,
which is the limit. "
"Stop searching and write your one or two sentence summary now."
)
if any(f["claim"].lower() == claim.lower() for f in findings):
return "Not saved: you already recorded this exact claim."

findings.append({"claim": claim, "source_url": url,
"source_title": title, "subtask": subtask_title})
remaining = config.MAX_FINDINGS_PER_RESEARCHER - len(findings)
return f"Saved finding {len(findings)}. You can record up to
{remaining} more."
```

scout/agents.py

```
"""Scout's four agents: the planner, the researchers, the writer
and the reviewer."""
import re

from . import config, llm
from .prompts import load_prompt
from .tools import (
RECORD_FINDING_TOOL,
SUBMIT_PLAN_TOOL,
SUBMIT_REVIEW_TOOL,
WEB_SEARCH_TOOL,
handle_record_finding,
)


def ask_for_tool(model: str, system: str, tool: dict, content:
str, max_tokens: int = 4000) -> dict:
"""Ask Claude to answer by calling one tool, and return what it
passed to that tool.

Some newer models don't allow tool_choice to force a particular
tool, so we
ask for it in the prompt and check the reply. If Claude answers
in prose
instead, we remind it once.
"""
messages = [{"role": "user", "content": content}]
for _attempt in range(2):
response = llm.client().messages.create(
model=model, max_tokens=max_tokens, system=system, tools=[tool],
messages=messages
)
for block in response.content:
if block.type == "tool_use" and block.name == tool["name"]:
return block.input
messages += [
{"role": "assistant", "content": response.content},
{"role": "user", "content": f"Please give your answer by calling
the {tool['name']} tool."},
]
raise RuntimeError(f"Claude did not call {tool['name']}")


def _text(response) -> str:
```

```
return "".join(block.text for block in response.content if
block.type == "text")


# ------------------------------------------------------------
- planner
def plan_research(question: str) -> list[dict]:
"""Split the question into independent research tasks."""
plan = ask_for_tool(
config.LEAD_MODEL, load_prompt("planner"), SUBMIT_PLAN_TOOL, f"
<question>{question}</question>"
)
subtasks = plan.get("subtasks", [])
if not subtasks:
raise RuntimeError("The planner returned an empty plan")
return subtasks[: config.MAX_SUBTASKS]


# ------------------------------------------------------------
researcher
def research_subtask(question: str, subtask: dict) -> list[dict]:
"""One researcher: an agent in a loop with web_search and
record_finding."""
findings: list[dict] = []
hints = ", ".join(subtask.get("search_hints", []))
messages = [
{
"role": "user",
"content": (
f"<overall_question>{question}</overall_question>\n"
f"<your_task title=\"
{subtask['title']}\">\n{subtask['objective']}\n</your_task>\n"
f"<search_hints>{hints}</search_hints>"
),
}
]

for _turn in range(config.MAX_RESEARCH_TURNS):
response = llm.client().messages.create(
model=config.WORKER_MODEL,
max_tokens=4000,
system=load_prompt("researcher"),
tools=[WEB_SEARCH_TOOL, RECORD_FINDING_TOOL],
messages=messages,
)
messages.append({"role": "assistant", "content":
```

```
response.content})

if response.stop_reason == "tool_use":
# Claude wants to use one of OUR tools. Run it and send back the
result.
results = []
for block in response.content:
if block.type != "tool_use":
continue
if block.name == "record_finding":
output = handle_record_finding(block.input, findings,
subtask["title"])
else:
output = f"Unknown tool: {block.name}"
results.append({"type": "tool_result", "tool_use_id": block.id,
"content": output})
messages.append({"role": "user", "content": results})
continue

if response.stop_reason == "pause_turn":
# A long server-side search paused. Sending the conversation back
lets it carry on.
continue

break # "end_turn" (finished) or "max_tokens"

return findings


# ----------------------------------------------------------------
-- writer
def number_sources(findings: list[dict]) -> list[dict]:
"""Give every distinct web page a number, in the order it was
found."""
sources: list[dict] = []
seen: dict[str, int] = {}
for f in findings:
if f["source_url"] not in seen:
seen[f["source_url"]] = len(sources) + 1
sources.append({"n": len(sources) + 1, "url": f["source_url"],
"title": f["source_title"]})
f["source_n"] = seen[f["source_url"]]
return sources


def format_findings(findings: list[dict]) -> str:
```

```
lines = [f'<finding source="[{f["source_n"]}]">{f["claim"]}
</finding>' for f in findings]
return "<findings>\n" + "\n".join(lines) + "\n</findings>"


def write_report(question: str, findings: list[dict], feedback:
list[str] | None = None) → tuple[str, list[dict]]:
"""Turn findings into a cited Markdown report. Returns (report,
sources)."""
sources = number_sources(findings)
if not findings:
return ("Scout could not find reliable information to answer this
question.", [])

content = f"<question>{question}
</question>\n{format_findings(findings)}"
if feedback:
content += "\n<reviewer_feedback>\n" + "\n".join(f"- {p}" for p
in feedback) + "\n</reviewer_feedback>"
content += "\nRewrite the report to fix every problem the
reviewer listed."

response = llm.client().messages.create(
model=config.LEAD_MODEL,
max_tokens=6000,
system=load_prompt("writer"),
messages=[{"role": "user", "content": content}],
)
text = _text(response)
match = re.search(r"<report>(.*?)</report>", text, re.DOTALL)
body = (match.group(1) if match else text).strip()

source_list = "\n".join(f"{s['n']}. [{s['title']}]({s['url']})"
for s in sources)
return f"{body}\n\n## Sources\n\n{source_list}\n", sources


# ---------------------------------------------------------------
reviewer
def check_citations(report: str, sources: list[dict]) →
list[str]:
"""Checks a computer can do without an AI: do the citations point
at real sources?"""
body = report.split("## Sources")[0]
cited = {int(n) for n in re.findall(r"\[(\d+)\]", body)}
problems = []
```

```
if sources and not cited:
problems.append("The report has no citations. Every factual
sentence needs one, like [1].")
bad = sorted(n for n in cited if n < 1 or n > len(sources))
if bad:
problems.append(f"These citation numbers do not match any source:
{bad}. Use 1 to {len(sources)} only.")
return problems


def review_report(question: str, report: str, findings:
list[dict], sources: list[dict]) → dict:
"""The evaluator half of the evaluator-optimizer loop."""
problems = check_citations(report, sources)
if not sources:
return {"approved": True, "problems": []}
number_sources(findings) # make sure every finding carries its
source number

verdict = ask_for_tool(
config.LEAD_MODEL,
load_prompt("reviewer"),
SUBMIT_REVIEW_TOOL,
f"<question>{question}
</question>\n{format_findings(findings)}\n<draft>\n{report}\n</dr
aft>",
)
problems += list(verdict.get("problems", []))
return {"approved": bool(verdict.get("approved")) and not
problems, "problems": problems}
```

scout/graph.py

```
"""Scout's workflow, drawn as a LangGraph graph.

START → plan → research (one copy per task, in parallel) →
write → review
^ |
+--------+ (if rejected)
"""
import operator
from typing import Annotated, Callable, TypedDict

from langgraph.graph import END, START, StateGraph
from langgraph.types import Send

from . import agents, config


class ScoutState(TypedDict, total=False):
question: str
subtasks: list[dict]
findings: Annotated[list[dict], operator.add] # each researcher
ADDS to this list
report: str
sources: list[dict]
review: dict
revisions: int


class ResearchJob(TypedDict):
question: str
subtask: dict


def plan_node(state: ScoutState) → dict:
return {"subtasks": agents.plan_research(state["question"]),
"revisions": 0}


def send_to_researchers(state: ScoutState) → list[Send]:
# One Send per task: LangGraph runs all the research nodes at the
same time.
return [Send("research", {"question": state["question"],
"subtask": t}) for t in state["subtasks"]]
```

```
def research_node(job: ResearchJob) → dict:
return {"findings": agents.research_subtask(job["question"],
job["subtask"])}


def write_node(state: ScoutState) → dict:
feedback = state["review"]["problems"] if state.get("review")
else None
report, sources = agents.write_report(state["question"],
state["findings"], feedback)
return {"report": report, "sources": sources}


def review_node(state: ScoutState) → dict:
review = agents.review_report(state["question"], state["report"],
state["findings"], state["sources"])
return {"review": review, "revisions": state.get("revisions", 0)
+ 1}


def after_review(state: ScoutState) → str:
if state["review"]["approved"] or state["revisions"] >
config.MAX_REVISIONS:
return END
return "write"


def build_graph():
builder = StateGraph(ScoutState)
builder.add_node("plan", plan_node)
builder.add_node("research", research_node)
builder.add_node("write", write_node)
builder.add_node("review", review_node)

builder.add_edge(START, "plan")
builder.add_conditional_edges("plan", send_to_researchers,
["research"])
builder.add_edge("research", "write")
builder.add_edge("write", "review")
builder.add_conditional_edges("review", after_review, ["write",
END])
return builder.compile()


graph = build_graph()
```

```
def describe(node: str, update: dict) → str:
"""Turn a graph step into a short progress message for the web
page."""
if node == "plan":
titles = "; ".join(t["title"] for t in update["subtasks"])
return f"Planned {len(update['subtasks'])} research task(s):
{titles}"
if node == "research":
return f"A researcher finished with {len(update['findings'])}
finding(s)"
if node == "write":
return "Wrote a draft report"
if node == "review":
if update["review"]["approved"]:
return "Fact-checker approved the report"
return f"Fact-checker asked for {len(update['review']
['problems'])} fix(es)"
return node


def run_research(question: str, on_progress: Callable[[str],
None] | None = None) → dict:
"""Run the whole graph for one question and return the final
state."""
state: dict = {"question": question, "findings": []}
for step in graph.stream({"question": question, "findings": []},
stream_mode="updates"):
for node, update in step.items():
if not update:
continue
if node == "research":
state["findings"] = state["findings"] + update["findings"]
else:
state.update(update)
if on_progress:
on_progress(describe(node, update))
return state
```

scout/store.py

```
"""Saves research jobs and their reports in a small SQLite
database file."""
import json
import sqlite3
import uuid
from datetime import datetime, timezone

from . import config


def _connect():
conn = sqlite3.connect(config.DB_PATH)
conn.row_factory = sqlite3.Row
conn.execute(
"""CREATE TABLE IF NOT EXISTS jobs (
id TEXT PRIMARY KEY,
question TEXT NOT NULL,
status TEXT NOT NULL,
progress TEXT NOT NULL DEFAULT '[]',
report TEXT,
error TEXT,
created_at TEXT NOT NULL
)"""
)
return conn


def _now() -> str:
return datetime.now(timezone.utc).isoformat(timespec="seconds")


def create_job(question: str) -> str:
job_id = uuid.uuid4().hex[:12]
with _connect() as conn:
conn.execute(
"INSERT INTO jobs (id, question, status, created_at) VALUES (?,
?, 'running', ?)",
(job_id, question, _now()),
)
return job_id


def add_progress(job_id: str, message: str) -> None:
with _connect() as conn:
```

```
row = conn.execute("SELECT progress FROM jobs WHERE id = ?",
(job_id,)).fetchone()
progress = json.loads(row["progress"]) + [message]
conn.execute("UPDATE jobs SET progress = ? WHERE id = ?",
(json.dumps(progress), job_id))


def finish_job(job_id: str, report: str) -> None:
with _connect() as conn:
conn.execute("UPDATE jobs SET status = 'done', report = ? WHERE
id = ?", (report, job_id))


def fail_job(job_id: str, error: str) -> None:
with _connect() as conn:
conn.execute("UPDATE jobs SET status = 'failed', error = ? WHERE
id = ?", (error, job_id))


def get_job(job_id: str) -> dict | None:
with _connect() as conn:
row = conn.execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
if row is None:
return None
job = dict(row)
job["progress"] = json.loads(job["progress"])
return job


def list_jobs(limit: int = 20) -> list[dict]:
with _connect() as conn:
rows = conn.execute(
"SELECT id, question, status, created_at FROM jobs ORDER BY
created_at DESC LIMIT ?", (limit,)
).fetchall()
return [dict(r) for r in rows]


def count_jobs_today() -> int:
today = datetime.now(timezone.utc).date().isoformat()
with _connect() as conn:
row = conn.execute("SELECT COUNT(*) AS n FROM jobs WHERE
created_at >= ?", (today,)).fetchone()
return row["n"]
```

scout/server.py

```
"""Scout's web server. Start it with: uvicorn scout.server:app --
reload"""
import secrets
from pathlib import Path

from fastapi import BackgroundTasks, Depends, FastAPI,
HTTPException
from fastapi.responses import FileResponse
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from pydantic import BaseModel

from . import config, store
from .graph import run_research

app = FastAPI(title="Scout")
STATIC = Path(__file__).resolve().parent / "static"
basic = HTTPBasic(auto_error=False)


def require_password(credentials: HTTPBasicCredentials | None =
Depends(basic)):
"""If SCOUT_PASSWORD is set, ask the browser for it (username:
scout)."""
if not config.PASSWORD:
return
ok = credentials is not None and secrets.compare_digest(
credentials.username.encode() + b":" +
credentials.password.encode(),
b"scout:" + config.PASSWORD.encode(),
)
if not ok:
raise HTTPException(status_code=401, headers={"WWW-Authenticate":
"Basic"})


class Question(BaseModel):
question: str


def run_job(job_id: str, question: str) -> None:
try:
state = run_research(question, on_progress=lambda msg:
store.add_progress(job_id, msg))
store.finish_job(job_id, state["report"])
```

```
except Exception as exc: # show the error on the page rather than
failing silently
store.fail_job(job_id, f"{type(exc).__name__}: {exc}")


@app.get("/health")
def health():
return {"ok": True}


@app.get("/", dependencies=[Depends(require_password)])
def home():
return FileResponse(STATIC / "index.html")


@app.post("/api/research", dependencies=
[Depends(require_password)])
def start_research(body: Question, background: BackgroundTasks):
question = body.question.strip()
if not 10 ⩽ len(question) ⩽ 500:
raise HTTPException(400, "Please write a question between 10 and
500 characters long.")
if store.count_jobs_today() ⩾ config.DAILY_LIMIT:
raise HTTPException(429, "Scout has reached today's question
limit. Try again tomorrow.")
job_id = store.create_job(question)
background.add_task(run_job, job_id, question)
return {"id": job_id}


@app.get("/api/research/{job_id}", dependencies=
[Depends(require_password)])
def get_research(job_id: str):
job = store.get_job(job_id)
if job is None:
raise HTTPException(404, "No such research job.")
return job


@app.get("/api/history", dependencies=
[Depends(require_password)])
def history():
return store.list_jobs()
```

scout/static/index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-
scale=1">
<title>Scout</title>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/marked/12.0.2/marked.
min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/dompurify/3.1.6/purif
y.min.js"></script>
<style>
:root { --ink: #1d1d1b; --muted: #6b6b66; --line: #e2e0d8; --
paper: #fbfaf6; --accent: #2f5d50; }
@media (prefers-color-scheme: dark) {
:root { --ink: #ecebe6; --muted: #a3a29b; --line: #34332f; --
paper: #191917; --accent: #7fb8a4; }
}
* { box-sizing: border-box; }
body { margin: 0; background: var(--paper); color: var(--ink);
font: 17px/1.6 Georgia, "Iowan Old Style", serif; }
main { max-width: 760px; margin: 0 auto; padding: 32px 16px 80px;
}
h1 { font-size: 30px; margin: 0 0 4px; letter-spacing: -0.5px; }
.lede { color: var(--muted); margin: 0 0 24px; }
textarea { width: 100%; min-height: 96px; padding: 12px; font:
inherit; color: inherit;
background: transparent; border: 1px solid var(--line); border-
radius: 8px; }
button { margin-top: 10px; padding: 10px 18px; font: 600 15px
system-ui, sans-serif;
color: var(--paper); background: var(--accent); border: 0;
border-radius: 8px; cursor: pointer; }
button:disabled { opacity: 0.5; cursor: wait; }
#progress { list-style: none; padding: 0; margin: 24px 0; font:
14px/1.5 system-ui, sans-serif; color: var(--muted); }
#progress li::before { content: "› "; color: var(--accent); }
#report { border-top: 1px solid var(--line); padding-top: 8px; }
#report a { color: var(--accent); }
.error { color: #b3261e; font-family: system-ui, sans-serif; }
h2.small { font: 600 13px system-ui, sans-serif; text-transform:
uppercase; letter-spacing: 1px;
```

```
color: var(--muted); margin-top: 48px; }
#history { padding-left: 18px; font-size: 15px; }
#history a { color: inherit; }
</style>
</head>
<body>
<main>
<h1>Scout</h1>
<p class="lede">Ask a question. A team of research agents will
search the web and write you a cited answer.</p>

<form id="ask">
<textarea id="question" placeholder="For example: How do heat
pumps compare with gas boilers on running costs in the UK?"
required></textarea>
<button id="go" type="submit">Research it</button>
</form>

<ul id="progress"></ul>
<article id="report"></article>

<h2 class="small">Recent questions</h2>
<ol id="history"></ol>
</main>

<script>
const $ = (id) => document.getElementById(id);
let timer = null;

async function api(path, options) {
const res = await fetch(path, options);
const data = await res.json().catch(() => ({}));
if (!res.ok) throw new Error(data.detail || `Request failed
(${res.status})`);
return data;
}

function show(job) {
$("progress").innerHTML = job.progress.map((p) =>
`<li>${p.replace(/</g, "&lt;")}</li>`).join("");
if (job.status === "done") {
$("report").innerHTML =
DOMPurify.sanitize(marked.parse(job.report));
} else if (job.status === "failed") {
$("report").innerHTML = `<p class="error">Something went wrong:
${job.error.replace(/</g, "&lt;")}</p>`;
```

```
} else {
$("report").innerHTML = "<p><em>Researching… this usually takes
one to four minutes.</em></p>";
}
}

async function follow(id) {
clearInterval(timer);
const tick = async () => {
const job = await api(`/api/research/${id}`);
show(job);
if (job.status !== "running") {
clearInterval(timer);
$("go").disabled = false;
loadHistory();
}
};
await tick();
timer = setInterval(tick, 2000);
}

async function loadHistory() {
const jobs = await api("/api/history");
$("history").innerHTML = jobs.map((j) =>
`<li><a href="#${j.id}">${j.question.replace(/</g, "&lt;")}</a>
<small>(${j.status})</small></li>`).join("");
}

$("ask").addEventListener("submit", async (event) => {
event.preventDefault();
$("go").disabled = true;
$("progress").innerHTML = "";
try {
const { id } = await api("/api/research", {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ question: $("question").value }),
});
location.hash = id;
} catch (err) {
$("report").innerHTML = `<p class="error">${err.message}</p>`;
$("go").disabled = false;
}
});

window.addEventListener("hashchange", () => location.hash &&
```

```
follow(location.hash.slice(1)));
if (location.hash) follow(location.hash.slice(1));
loadHistory();
</script>
</body>
</html>
```

scout/mcp_server.py

```
"""Scout as an MCP server, so Claude Code or Claude Desktop can
use it as a tool.

Add it to Claude Code (from the scout folder, with the virtual
environment active):
claude mcp add scout -- "$(pwd)/.venv/bin/python"
"$(pwd)/scout/mcp_server.py"
"""
import sys
from pathlib import Path

# Make "import scout" work no matter which folder this file is
started from.
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from mcp.server.mcpserver import MCPServer # noqa: E402 (MCP
Python SDK 2.x)

from scout.graph import run_research # noqa: E402

mcp = MCPServer("scout")


@mcp.tool()
def deep_research(question: str) → str:
"""Research a question on the public web using Scout's team of
research agents.

Returns a Markdown report that answers the question, with
numbered citations
and a list of sources. Use this for questions that need several
up-to-date
sources, comparisons, or a written summary. Do not use it for
simple facts you
already know. It takes one to four minutes and costs a small
amount per call.

question: the full question, written as you would ask a human
researcher,
including any country, time period or other constraints.
"""
return run_research(question)["report"]
```

```
if __name__ == "__main__":
mcp.run() # talks to Claude over standard input and output
```

tests/fake_llm.py

```
"""A pretend Claude for tests. It gives scripted answers so tests
are fast, free and repeatable."""
from types import SimpleNamespace as NS


def text_block(text):
return NS(type="text", text=text)


def tool_block(name, tool_input, block_id="toolu_1"):
return NS(type="tool_use", name=name, input=tool_input,
id=block_id)


def reply(blocks, stop_reason="end_turn"):
return NS(content=blocks, stop_reason=stop_reason)


class FakeMessages:
def __init__(self, approve=True):
self.calls = []
self.approve = approve

def create(self, **kwargs):
self.calls.append(kwargs)
tool_names = [t.get("name") for t in kwargs.get("tools", [])]
# The only tool offered tells us which agent is calling.
forced = tool_names[0] if len(tool_names) == 1 else None

if forced == "submit_plan":
return reply([tool_block("submit_plan", {"subtasks": [
{"title": "Part A", "objective": "Find fact A.", "search_hints":
["fact a"]},
{"title": "Part B", "objective": "Find fact B.", "search_hints":
["fact b"]},
]})], "tool_use")

if forced == "submit_review":
if self.approve:
return reply([tool_block("submit_review", {"approved": True,
"problems": []})], "tool_use")
return reply([tool_block("submit_review", {"approved": False,
"problems": ["Cite [2] in paragraph one."]})], "tool_use")
```

```
if forced == "submit_grade":
scores = {k: 0.8 for k in ["factual_accuracy",
"citation_accuracy", "completeness", "source_quality",
"clarity"]}
return reply([tool_block("submit_grade", {**scores, "comment":
"Fine."})], "tool_use")

if "record_finding" in tool_names:
# First turn: record a finding. Second turn: finish.
already_recorded = any(m["role"] == "user" and
isinstance(m["content"], list) for m in kwargs["messages"])
if not already_recorded:
task = kwargs["messages"][0]["content"]
letter = "A" if "Part A" in task else "B"
return reply([tool_block("record_finding", {
"claim": f"Fact {letter} is true according to the source.",
"source_url": f"https://example.org/{letter.lower()}",
"source_title": f"Example {letter}",
})], "tool_use")
return reply([text_block("Found what I needed.")])

# Otherwise this is the writer.
return reply([text_block("<report>Fact A is true [1]. Fact B is
true [2].</report>")])


class FakeClient:
def __init__(self, approve=True):
self.messages = FakeMessages(approve)
```

tests/conftest.py

```
import pytest

from scout import config, llm
from tests.fake_llm import FakeClient


@pytest.fixture(autouse=True)
def fake_claude(tmp_path, monkeypatch):
"""Every test gets a pretend Claude and its own empty
database."""
monkeypatch.setattr(config, "DB_PATH", str(tmp_path / "test.db"))
monkeypatch.setattr(config, "PASSWORD", "")
client = FakeClient()
llm.set_client(client)
yield client
llm.set_client(None)
```

tests/test_tools.py

```
from scout import config
from scout.agents import check_citations
from scout.tools import handle_record_finding

GOOD = {"claim": "The bridge opened in 1937.", "source_url":
"https://example.org/bridge", "source_title": "Bridge"}


def test_good_finding_is_saved():
findings = []
message = handle_record_finding(GOOD, findings, "Bridge history")
assert message.startswith("Saved finding 1")
assert findings[0]["source_url"] == "https://example.org/bridge"


def test_bad_url_gets_a_helpful_error():
findings = []
message = handle_record_finding({**GOOD, "source_url":
"example.org"}, findings, "t")
assert "must be a full web address" in message
assert findings == []


def test_duplicate_is_refused():
findings = []
handle_record_finding(GOOD, findings, "t")
assert "already recorded" in handle_record_finding(GOOD,
findings, "t")


def test_limit_is_enforced(monkeypatch):
monkeypatch.setattr(config, "MAX_FINDINGS_PER_RESEARCHER", 1)
findings = []
handle_record_finding(GOOD, findings, "t")
other = {**GOOD, "claim": "A different fact about the bridge."}
assert "limit" in handle_record_finding(other, findings, "t")


def test_citation_check_catches_missing_sources():
sources = [{"n": 1}, {"n": 2}]
assert check_citations("Fact [1]. Fact [2].", sources) == []
assert check_citations("No citations here.", sources)
assert "[7]" in check_citations("Fact [7].", sources)[0]
```

tests/test_graph.py

```
from scout import llm
from scout.graph import run_research
from tests.fake_llm import FakeClient


def test_full_run_produces_cited_report():
progress = []
state = run_research("What are facts A and B?",
on_progress=progress.append)

assert len(state["subtasks"]) == 2
assert len(state["findings"]) == 2 # one from each parallel
researcher
assert "[1]" in state["report"] and "## Sources" in
state["report"]
assert "https://example.org/a" in state["report"]
assert state["review"]["approved"] is True
assert progress[0].startswith("Planned 2 research task(s)")


def test_rejected_report_is_rewritten_once():
client = FakeClient(approve=False)
llm.set_client(client)
state = run_research("What are facts A and B?")

writer_calls = [c for c in client.messages.calls if not
c.get("tools")]
assert len(writer_calls) == 2 # first draft plus one revision
assert "reviewer_feedback" in writer_calls[1]["messages"][0]
["content"]
assert state["revisions"] == 2
```

tests/test_server.py

```
from fastapi.testclient import TestClient

from scout import config
from scout.server import app

client = TestClient(app)


def test_question_runs_to_a_report():
started = client.post("/api/research", json={"question": "What
are facts A and B?"})
assert started.status_code == 200
job = client.get(f"/api/research/{started.json()['id']}").json()
assert job["status"] == "done"
assert "## Sources" in job["report"]
assert client.get("/api/history").json()[0]["id"] ==
started.json()["id"]


def test_short_question_is_rejected():
assert client.post("/api/research", json={"question":
"hi"}).status_code == 400


def test_password_protects_the_app(monkeypatch):
monkeypatch.setattr(config, "PASSWORD", "s3cret")
assert client.get("/api/history").status_code == 401
assert client.get("/api/history", auth=("scout",
"s3cret")).status_code == 200
assert client.get("/health").status_code == 200


def test_daily_limit(monkeypatch):
monkeypatch.setattr(config, "DAILY_LIMIT", 1)
assert client.post("/api/research", json={"question": "What are
facts A and B?"}).status_code == 200
assert client.post("/api/research", json={"question": "What are
facts A and B?"}).status_code == 429
```

tests/test_eval.py

```
from evals.run_eval import run_one


def test_eval_grades_a_report():
result = run_one({"question": "What are facts A and B?",
"must_cover": ["A", "B"]})
assert result["overall"] == 0.8
```

evals/questions.jsonl (create by hand)

```
{"question": "What is the current population of Iceland, and how
has it changed over the last ten years?", "must_cover": ["a
recent population figure", "the direction and rough size of
change over ten years"]}
{"question": "How do heat pumps compare with gas boilers on
running costs and carbon emissions for a typical UK home?",
"must_cover": ["running cost comparison", "carbon emissions
comparison", "UK-specific context"]}
{"question": "What are the main differences between the Python
web frameworks FastAPI, Flask and Django?", "must_cover":
["FastAPI", "Flask", "Django", "when to choose each"]}
{"question": "Who invented the World Wide Web, where, and in what
year was the first website published?", "must_cover": ["Tim
Berners-Lee", "CERN", "1991"]}
{"question": "What does the scientific evidence say about the
health effects of intermittent fasting compared with ordinary
calorie restriction?", "must_cover": ["weight loss comparison",
"other health markers", "strength or limits of the evidence"]}
{"question": "What were the main causes of the 2008 global
financial crisis?", "must_cover": ["subprime mortgages",
"leverage or securitisation", "bank failures such as Lehman
Brothers"]}
{"question": "Which European countries generate the largest share
of their electricity from wind power, and roughly what share is
it?", "must_cover": ["at least three countries", "approximate
percentages", "year of the data"]}
{"question": "How does the James Webb Space Telescope differ from
the Hubble Space Telescope?", "must_cover": ["wavelengths
(infrared vs visible/ultraviolet)", "mirror size", "orbit
location"]}
```

evals/run_eval.py (create by hand)

```
"""Scout's exam. Runs a set of questions and has Claude grade
every report.

python -m evals.run_eval # all questions
python -m evals.run_eval --limit 3 # just the first three
(cheaper)

Prints a line like "SCORE: 0.812" and appends one row to
evals/results.tsv.
This file and questions.jsonl are the referee: agents must never
edit them.
"""
import argparse
import json
import subprocess
import time
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

from scout import config, llm
from scout.agents import ask_for_tool, format_findings,
number_sources
from scout.graph import run_research

HERE = Path(__file__).resolve().parent

GRADE_TOOL = {
"name": "submit_grade",
"description": "Submit scores for the research report.",
"input_schema": {
"type": "object",
"properties": {
"factual_accuracy": {"type": "number", "description": "0.0-1.0:
do the report's claims match the findings?"},
"citation_accuracy": {"type": "number", "description": "0.0-1.0:
does each citation support its sentence?"},
"completeness": {"type": "number", "description": "0.0-1.0: are
all must_cover points answered?"},
"source_quality": {"type": "number", "description": "0.0-1.0:
primary, reputable sources rather than content farms?"},
"clarity": {"type": "number", "description": "0.0-1.0: does it
lead with the answer and read clearly?"},
"comment": {"type": "string", "description": "One sentence on the
biggest weakness."},
```

```
},
"required": ["factual_accuracy", "citation_accuracy",
"completeness", "source_quality", "clarity", "comment"],
},
}

JUDGE_PROMPT = """You are a strict but fair examiner grading a
research report written by an AI research assistant.
You will see the question, the points a good answer must cover,
the findings the researchers collected
(with their sources), and the final report. Grade each criterion
from 0.0 to 1.0. Judge citation and
factual accuracy only against the findings. Call submit_grade."""


def grade(item: dict, state: dict) → dict:
number_sources(state["findings"])
sources = "\n".join(f"[{s['n']}] {s['title']} - {s['url']}" for s
in state.get("sources", []))
scores = ask_for_tool(config.LEAD_MODEL, JUDGE_PROMPT,
GRADE_TOOL, (
f"<question>{item['question']}</question>\n"
f"<must_cover>{json.dumps(item['must_cover'])}</must_cover>\n"
f"
{format_findings(state['findings'])}\n<sources>\n{sources}\n</sou
rces>\n"
f"<report>\n{state['report']}\n</report>"))
keys = ["factual_accuracy", "citation_accuracy", "completeness",
"source_quality", "clarity"]
scores["overall"] = round(sum(float(scores[k]) for k in keys) /
len(keys), 3)
return scores


def run_one(item: dict) → dict:
started = time.time()
try:
state = run_research(item["question"])
scores = grade(item, state)
except Exception as exc:
scores = {"overall": 0.0, "comment": f"crashed: {exc}"}
scores["seconds"] = round(time.time() - started)
scores["question"] = item["question"]
return scores
```

```
def main():
parser = argparse.ArgumentParser()
parser.add_argument("--limit", type=int, default=None, help="only
run the first N questions")
args = parser.parse_args()

items = [json.loads(line) for line in (HERE /
"questions.jsonl").read_text().splitlines() if line.strip()]
items = items[: args.limit] if args.limit else items

with ThreadPoolExecutor(max_workers=3) as pool:
results = list(pool.map(run_one, items))

for r in results:
print(f"{r['overall']:.2f} {r['seconds']:>4}s {r['question']
[:70]} -- {r.get('comment', '')}")
score = round(sum(r["overall"] for r in results) / len(results),
3)
print(f"\nSCORE: {score}")

try:
commit = subprocess.run(["git", "rev-parse", "--short", "HEAD"],
capture_output=True, text=True).stdout.strip()
except OSError:
commit = ""
log = HERE / "results.tsv"
if not log.exists():
log.write_text("time\tcommit\tquestions\tscore\n")
with log.open("a") as f:
f.write(f"{time.strftime('%Y-%m-%d %H:%M')}\t{commit or '-
'}\t{len(results)}\t{score}\n")


if __name__ == "__main__":
main()
```

.claude/settings.json

As printed in Chapter 6. On Windows, change `"python3"` to `"python"` in both places.

.claude/hooks/protect.py

```
#!/usr/bin/env python3
"""PreToolUse hook: a check that can say no.

Claude Code runs this before every Edit, Write or Bash action and
sends the
details as JSON on standard input. If we print a "deny" decision,
the action
is blocked and Claude is told why.
"""
import json
import sys
from pathlib import PurePath

PROTECTED_FILES = {".env"} # secrets
PROTECTED_PATHS = ("evals/questions.jsonl", "evals/run_eval.py")
# the referee
DANGEROUS_COMMANDS = ("rm -rf", "git push --force", "git push -
f", "git reset --hard origin")


def deny(reason: str) → None:
print(json.dumps({
"hookSpecificOutput": {
"hookEventName": "PreToolUse",
"permissionDecision": "deny",
"permissionDecisionReason": reason,
}
}))
sys.exit(0)


event = json.load(sys.stdin)
tool = event.get("tool_name", "")
details = event.get("tool_input", {})

if tool in ("Edit", "Write", "MultiEdit"):
path = PurePath(details.get("file_path", ""))
if path.name in PROTECTED_FILES:
deny(f"{path.name} holds secrets and must be edited by a human,
not by Claude.")
if any(str(path).replace("\\", "/").endswith(p) for p in
PROTECTED_PATHS):
deny(f"{path.name} is part of the evaluation. Agents may not
change how they are graded.")
```

```
if tool == "Bash":
command = details.get("command", "")
for bad in DANGEROUS_COMMANDS:
if bad in command:
deny(f"The command contains '{bad}', which is blocked in this
project. Ask the human to run it.")
if ".env" in command and any(word in command for word in ("cat ",
"less ", "head ", "tail ", "echo ")):
deny("Reading .env would print secret keys into the
conversation.")

sys.exit(0) # no objection: the normal permission rules apply
```

.claude/hooks/run_tests.py

```
#!/usr/bin/env python3
"""Stop hook: Claude may not say "done" while the tests are
failing.

Claude Code runs this when Claude is about to finish its turn.
Exit code 2
sends Claude back to work, with our message explaining why.
"""
import json
import subprocess
import sys

event = json.load(sys.stdin)
if event.get("stop_hook_active"):
sys.exit(0) # we already sent Claude back once this turn; don't
loop forever

result = subprocess.run(
[sys.executable, "-m", "pytest", "-q", "-x", "--no-header"],
capture_output=True, text=True, timeout=300,
)
if result.returncode == 0:
sys.exit(0)

tail = "\n".join(result.stdout.strip().splitlines()[-25:])
print(f"The test suite is failing. Fix it before
finishing.\n\n{tail}", file=sys.stderr)
sys.exit(2)
```

.claude/agents/code-reviewer.md

```
---
name: code-reviewer
description: Reviews recent code changes in Scout for bugs,
security problems and unclear code. Use after any change to files
in scout/ and before committing.
tools: Read, Grep, Glob, Bash
model: sonnet
---

You are a careful senior reviewer looking at changes to Scout, a
Python web app that calls the Claude API.
The person who owns this project is not a programmer, so explain
every finding in plain English.

1. Run `git diff` to see what changed since the last commit. Read
any changed file in full.
2. Check for, in order of importance:
- Secrets: API keys or passwords written into code, logs or the
web page.
- Cost risks: loops that could call Claude without a limit, or
limits removed from scout/config.py.
- Bugs: code that will crash, wrong variable names, unhandled
errors in the web server.
- Missing tests: new behaviour in scout/ with no matching test in
tests/.
- Clarity: names or code a newcomer could not follow.
3. Do not edit any files. You only report.

Reply with a short list grouped as "Must fix", "Should fix" and
"Fine as it is".
For each item give the file, the line, the problem, and the fix
in one or two sentences.
```

.claude/agents/eval-runner.md

```
---
name: eval-runner
description: Runs Scout's evaluation and reports only the score
and the weakest answers. Use when asked to evaluate, score or
benchmark Scout.
tools: Bash, Read
model: haiku
---

You run Scout's evaluation and summarise it, so the long output
stays out of the main conversation.

1. Run `python -m evals.run_eval --limit 3` unless you were told
a different limit.
2. Read the printed table and the final SCORE line. Read the last
few lines of evals/results.tsv
to find the previous score.
3. Reply in no more than eight lines: the new score, the previous
score, whether it went up or
down, and the two lowest-scoring questions with the grader's
comment for each.

Never edit any file.
```

.claude/skills/add-tool/SKILL.md

```
---
name: add-tool
description: Adds a new tool that Scout's researcher agents can
call (for example a Wikipedia lookup or a currency converter).
Use when the user asks to give Scout's researchers a new tool or
ability.
---

# Adding a tool to Scout's researchers

Follow these steps in order. Do not skip the test.

1. **Agree the tool's purpose in one sentence.** Ask the user if
it is unclear. One tool, one clear job.
Prefer a tool that does a whole task (`lookup_company_profile`)
over several thin ones
(`get_company`, `get_people`, `get_address`).

2. **Define it in `scout/tools.py`**, next to
`RECORD_FINDING_TOOL`, as a dictionary with
`name`, `description` and `input_schema`.
- Name: lowercase with underscores, starting with a verb, e.g.
`lookup_wikipedia`.
- Description: write it for a new colleague. Say what it does,
when to use it, when NOT to use
it, and what it returns. Two to five sentences.
- Inputs: unambiguous names (`country_code`, not `country`), each
with a description and an example.

3. **Write the handler** in `scout/tools.py` as `handle_<name>
(tool_input, ...) → str`.
- Return only what the agent needs, as short readable text. No
raw IDs or huge JSON.
- Cap the size of what comes back (about 2,000 characters) and
say when you have cut it short.
- On bad input, return a message saying exactly what was wrong
and how to fix it, with an example.
- Use a timeout of 10 seconds on any web request.

4. **Wire it into the researcher** in `scout/agents.py`: add the
tool to the `tools=[...]` list in
`research_subtask`, and add a branch in the `if block.name ==
...` section that calls the handler.
```

```
5. **Mention it in `scout/prompts/researcher.md`** in the list of
tools, with one line on when to use it.

6. **Test it** in `tests/test_tools.py`: one test for a good call
and one for a bad input. Tests must
not touch the internet; replace any web request with a fake.

7. Run `python -m pytest -q` and show the user the result. Then
summarise in plain English what the
new tool does and how to try it.
```

.claude/skills/ship/SKILL.md

```
---
name: ship
description: Pre-release checklist for Scout. Runs tests, reviews
the changes, commits and pushes so the hosted copy redeploys.
disable-model-invocation: true
allowed-tools: Bash(python -m pytest *) Bash(git status *)
Bash(git diff *) Bash(git add *) Bash(git commit *)
---

# Ship Scout

Work through this checklist. Stop and tell the user if any step
fails.

## Current state
!`git status --short`

## Steps
1. Run `python -m pytest -q`. Every test must pass.
2. Use the code-reviewer subagent on the uncommitted changes. If
it reports anything under
"Must fix", stop and show the user.
3. Check `scout/config.py` still has every limit (MAX_SUBTASKS,
MAX_RESEARCH_TURNS,
MAX_SEARCHES_PER_RESEARCHER, MAX_FINDINGS_PER_RESEARCHER,
MAX_REVISIONS) and that none has
been raised above its original value without the user saying so.
4. Make sure no file being committed contains an API key (search
the diff for `sk-ant-`).
5. Stage and commit with a one-line message describing the change
for a non-programmer.
6. Tell the user the commit is ready and that they should run
`git push` themselves to deploy.
Do not push.

$ARGUMENTS
```

harness/feature_list.json

```
[
{"id": 1, "feature": "Download report as Markdown",
"steps": ["Finished report page shows a 'Download' button",
"Clicking it saves a .md file named after the question", "Test
covers the new /api/research/{id}/download endpoint"],
"passes": false},
{"id": 2, "feature": "Delete a past question from history",
"steps": ["Each history item has a small 'remove' link", "DELETE
/api/research/{id} removes it from the database", "Test covers
deleting and then getting a 404"],
"passes": false},
{"id": 3, "feature": "Show how long the research took",
"steps": ["Database stores a finished_at time", "Finished report
shows 'Researched in N seconds'", "Test checks finished_at is set
when a job completes"],
"passes": false},
{"id": 4, "feature": "Show findings count and sources count",
"steps": ["Final progress line reads 'N findings from M
sources'", "Test checks the message"],
"passes": false},
{"id": 5, "feature": "Example questions on the empty page",
"steps": ["When no report is showing, three clickable example
questions appear under the box", "Clicking one fills the question
box"],
"passes": false},
{"id": 6, "feature": "Friendly message when the daily limit is
reached",
"steps": ["The page shows the 429 message in plain words instead
of an error code", "Test checks the API message text"],
"passes": false},
{"id": 7, "feature": "Copy report button",
"steps": ["Finished report has a 'Copy' button that copies the
Markdown to the clipboard", "Button briefly says 'Copied'"],
"passes": false},
{"id": 8, "feature": "Mark failed jobs on restart",
"steps": ["When the server starts, any job still marked 'running'
is marked 'failed' with the message 'Scout restarted before this
finished'", "Test covers it"],
"passes": false}
]
```

harness/progress.md

```
# Scout harness progress log

Each session adds a short entry at the bottom: date, feature id,
what was done, what is left.

## Session 0 (set-up)
- Harness created. All features in harness/feature_list.json
start as "passes": false.
```

harness/init.sh

```
#!/usr/bin/env bash
# Gets a fresh session ready to work: installs packages and
proves the tests pass.
set -e
cd "$(dirname "$0")/.."
source .venv/bin/activate
pip install -q -r requirements.txt
python -m pytest -q
echo "Ready. Start the app with: uvicorn scout.server:app --
reload"
```

In Git Bash on Windows, change the activate line to `source .venv/Scripts/activate`.

harness/coding_prompt.md

```
You are one shift in a relay of engineers working on Scout. You
remember nothing from earlier shifts;
the files below are how the shifts talk to each other.

1. Get your bearings. Run `pwd`, read `harness/progress.md`, run
`git log --oneline -10`,
and read `harness/feature_list.json`.
2. Run `bash harness/init.sh`. If the tests fail, fixing them is
your whole job this shift.
3. Choose ONE feature: the lowest id whose "passes" is false.
Work on that feature only.
4. Build it, with a test that uses the fake client in tests/. Run
`python -m pytest -q` until it passes.
5. Check the feature against every item in its "steps" list.
Start the app and look at it if the
feature changes the page.
6. Only when every step is met, change that feature's "passes" to
true. Never edit any other field,
and never remove or reword features.
7. Commit with a message like "Feature 3: show research time".
8. Add an entry to the bottom of `harness/progress.md`: the
feature, what you did, anything the next
shift should know. Commit that too.
9. Stop. Do not start a second feature.
```

harness/run.sh

```
#!/usr/bin/env bash
# Runs up to N shifts of Claude Code, one feature per shift.
Usage: bash harness/run.sh 3
cd "$(dirname "$0")/.."
SHIFTS=${1:-1}
for i in $(seq 1 "$SHIFTS"); do
if ! grep -q '"passes": false' harness/feature_list.json; then
echo "Every feature passes. Nothing left to do."; break
fi
echo "═══ Shift $i of $SHIFTS ═══"
claude -p "$(cat harness/coding_prompt.md)" \
--permission-mode acceptEdits \
--allowedTools "Bash(python *)" "Bash(bash harness/init.sh)"
"Bash(git *)" "Bash(pwd)" "Bash(uvicorn *)"
done
```

program.md

```
# Scout autoresearch

You are improving Scout's prompts by experiment, the way
Karpathy's autoresearch improves a training
script. The human may be away. Work through the loop below
without asking whether to continue,
until you have run the number of experiments you were given
(default: 5). Then stop and report.

## What you may change
- ONLY the files in `scout/prompts/` (planner.md, researcher.md,
writer.md, reviewer.md).
- Nothing else. Not the code, not the tests, and never anything
in `evals/`.

## Setup (once)
1. Make sure the working tree is clean (`git status`). Create a
branch: `git checkout -b autoresearch/<today's date>`.
2. Run the baseline: `python -m evals.run_eval --limit 4`. Note
the SCORE. That is the score to beat.

## The loop (one experiment)
1. Read the latest results in `evals/results.tsv` and the
grader's comments from the last run.
Form ONE hypothesis about a prompt change that would raise the
score. Write it down in one sentence.
2. Make that single change to one prompt file. Keep it small.
3. Run `python -m pytest -q` (must pass), then `python -m
evals.run_eval --limit 4`.
4. If SCORE is higher than the best so far: `git commit -am
"experiment: <hypothesis> (score X)"`.
This is the new best.
5. If SCORE is the same or lower: `git checkout --
scout/prompts/` to throw the change away.
6. Append one line to `evals/experiments.md`: the hypothesis, the
score, kept or discarded.

## Taste
- A prompt that gets shorter and scores the same is an
improvement. Keep it.
- A tiny gain that makes a prompt much longer or more convoluted
is not worth keeping.
- Scores move a little between runs by chance. Treat a change of
less than 0.02 as no change.
```

```
## When you finish
Report the starting score, the best score, and the list of kept
changes, in plain English.
```

Appendix E. Sources and further reading

Every chapter draws on the material below. The ideas are theirs; any mistakes in the retelling are mine. Read the originals once you have finished the book: they will make more sense now.

Source	Used in	What to read it for
Building effective agents, Anthropic	Chapters 1, 9	Workflows versus agents; the five patterns; when not to build an agent
How we built our multi-agent research system, Anthropic	Chapters 1, 3, 8, 10, 14, 15	Orchestrator and subagents; eight prompting principles; evaluation with a model as judge
Effective harnesses for long-running agents, Anthropic	Chapter 13	Initializer and coding agents; feature lists and progress files
Writing effective tools for agents, Anthropic	Chapters 7, 12, 14	Tool choice, naming, responses, errors and descriptions; improving tools with evaluations
Claude Code 101, Anthropic Academy	Chapters 2, 4	A guided first course in Claude Code
Claude Code in action, Anthropic Academy	Chapter 4	Working habits: context, planning, hooks and integrations
Prompt engineering overview, Claude docs	Chapter 3	The full set of prompting techniques, with examples
Subagents, Claude Code docs	Chapter 10	Every subagent setting, including memory and worktrees
Hooks reference, Claude Code docs	Chapter 6	Every hook event and the exact input and output of each

Source	Used in	What to read it for
CLAUDE.md and memory, Claude Code docs	Chapter 5	Where memory files load from, rules, imports, auto memory
Skills, Claude Code docs	Chapter 11	All skill settings, arguments, dynamic context and evaluating skills
MCP, Claude Code docs	Chapter 12	Adding servers, scopes, authentication and limits
Web search tool, Claude docs	Chapter 7	Tool versions, domain filters and pricing
LangGraph, LangChain	Chapter 9	State, nodes, edges, Send, checkpointers and human-in-the-loop
karpathy/autoresearch, Andrej Karpathy	Chapter 14	The original agent-in-a-loop, and its `program.md`